# Doubts with Lakshay Bhaiya [Week 4]

Special class

Love Babbar • Sept 17, 2023

⇒ Vectors

# Array Level-3 [Join Here]

Special class

Love Babbar • Sept 11, 2023

→ 1D – Array

int arr[5];


int arr[] = {1, 2, 40, 50}

accessing
↳ 1 index

with function

reference

→ Arrays → Question

→ Array Level-2 → MD R.C

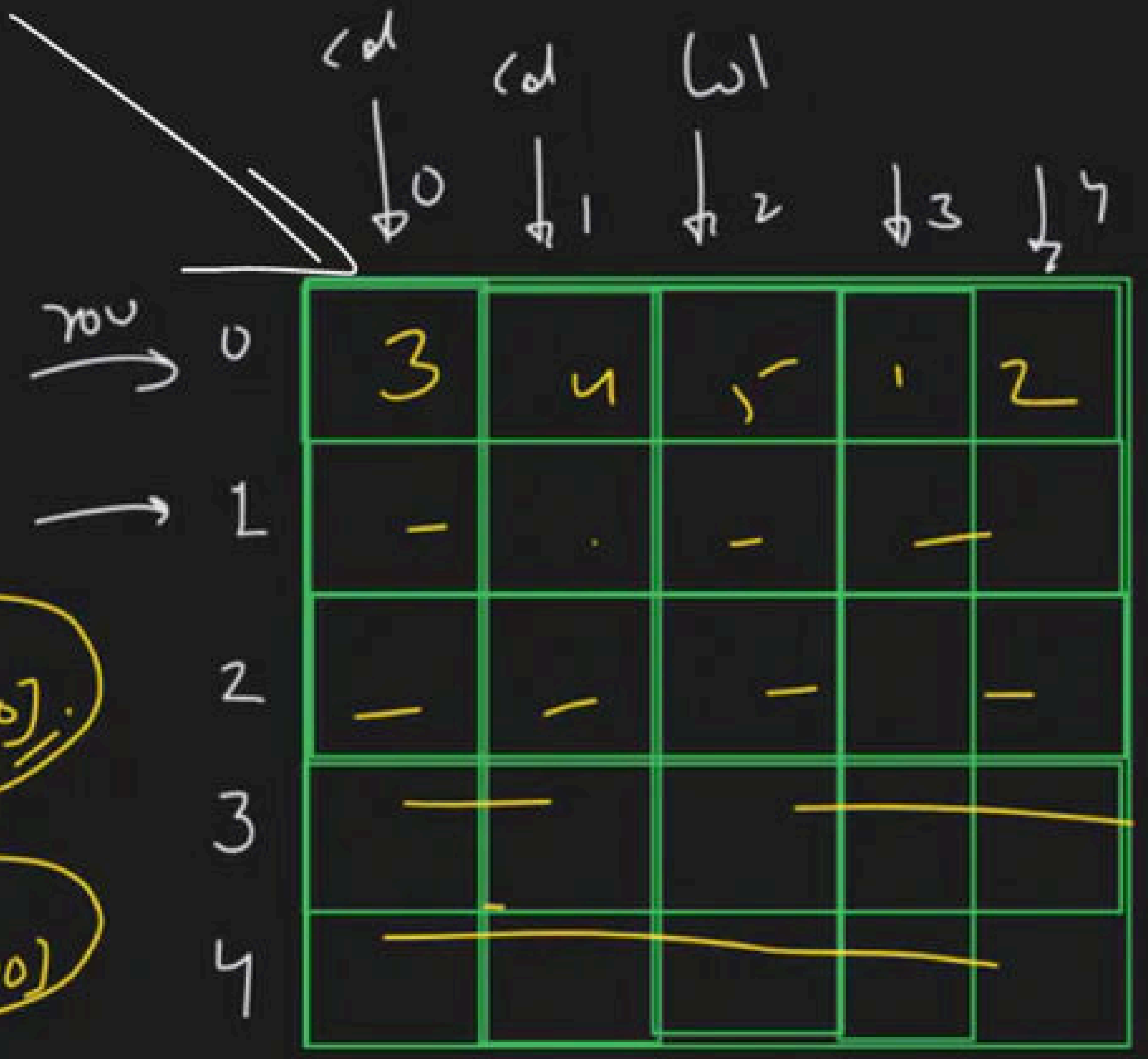
# 2D - Array

## Creation

1D → int arr(5);

2D → row → 5
col → 10 → int arr[5][10].

2D → 100 row
1000 col → int arr(100) [1000]




initialize

1D → int arr[] = {10, 20, 30}

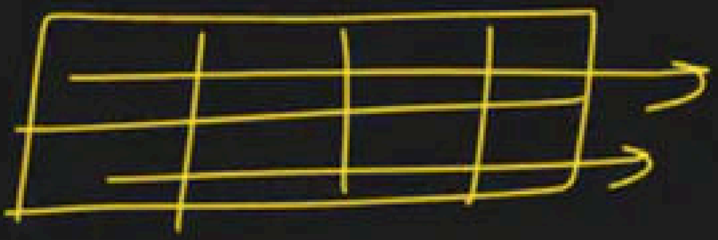
2D → row → 2 → int arr[2][4] = {

col → 4

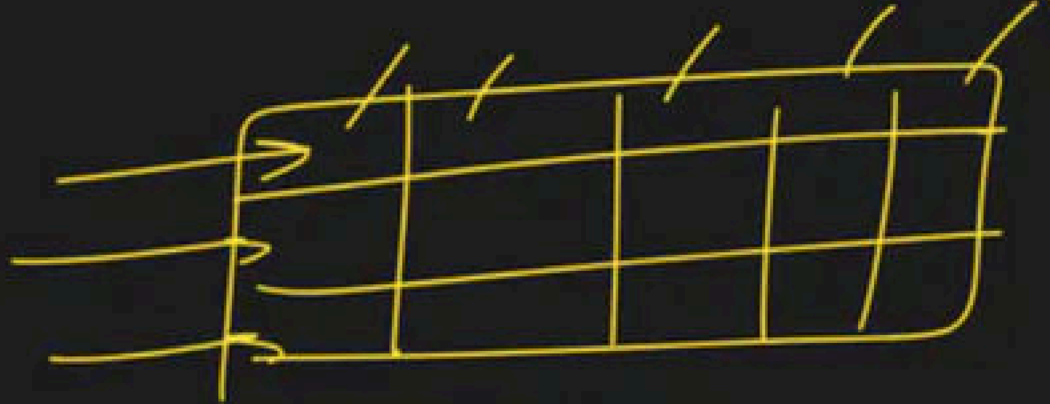
{10, 20, 30, 40},

{80, 70, 60, 50}

};



2D → row 3 row
5 cols



int arr[3][5]

= {

{1, 2, 3, 4, 5},

{6, 7, 8, 9, 10},

{10, 20, 30, 40, 50}

};

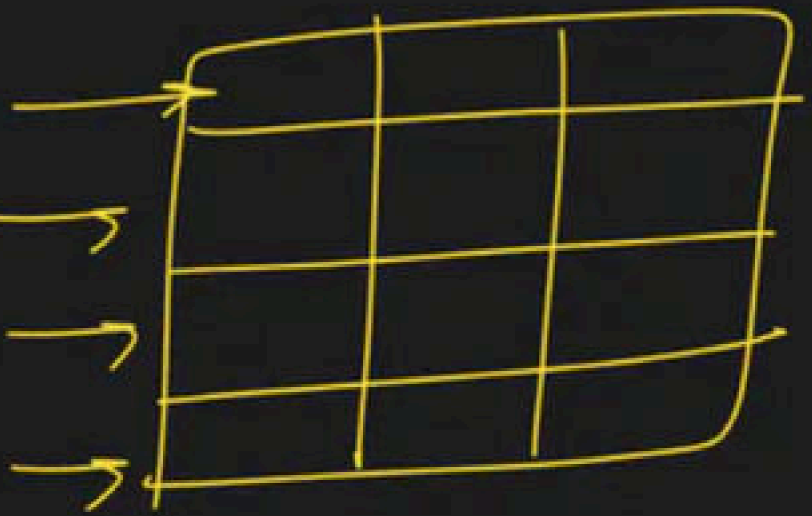
int a[ ][N]

2D → row → 4

col → 3

=

```
int arr[4][3]
=
{
{10, 20, 30},
{1, 2, 3},
{4, 6, 8},
{5, 7, 9}
}
```



## Access

- 1D → index → arr[i]

| 10 | 20 | 30 | 40 | 50 |
|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 |

arr[3] = 40

- 2D array → index

$\boxed{arr[i][j]}$

i = row index

j = col index

| | 0 | 1 | 2 |
|---|---|---|---|
| 0 | (0,0) 10 | (0,1) 20 | (0,2) 30 |
| 1 | (1,0) 40 | (1,1) 50 | (1,2) 60 |
| 2 | (2,0) 70 | (2,1) 80 | (2,2) 90 |

arr[0][0] = 10

arr[0][1] = 20

arr[0][2] = 30

arr[1][0] = 40

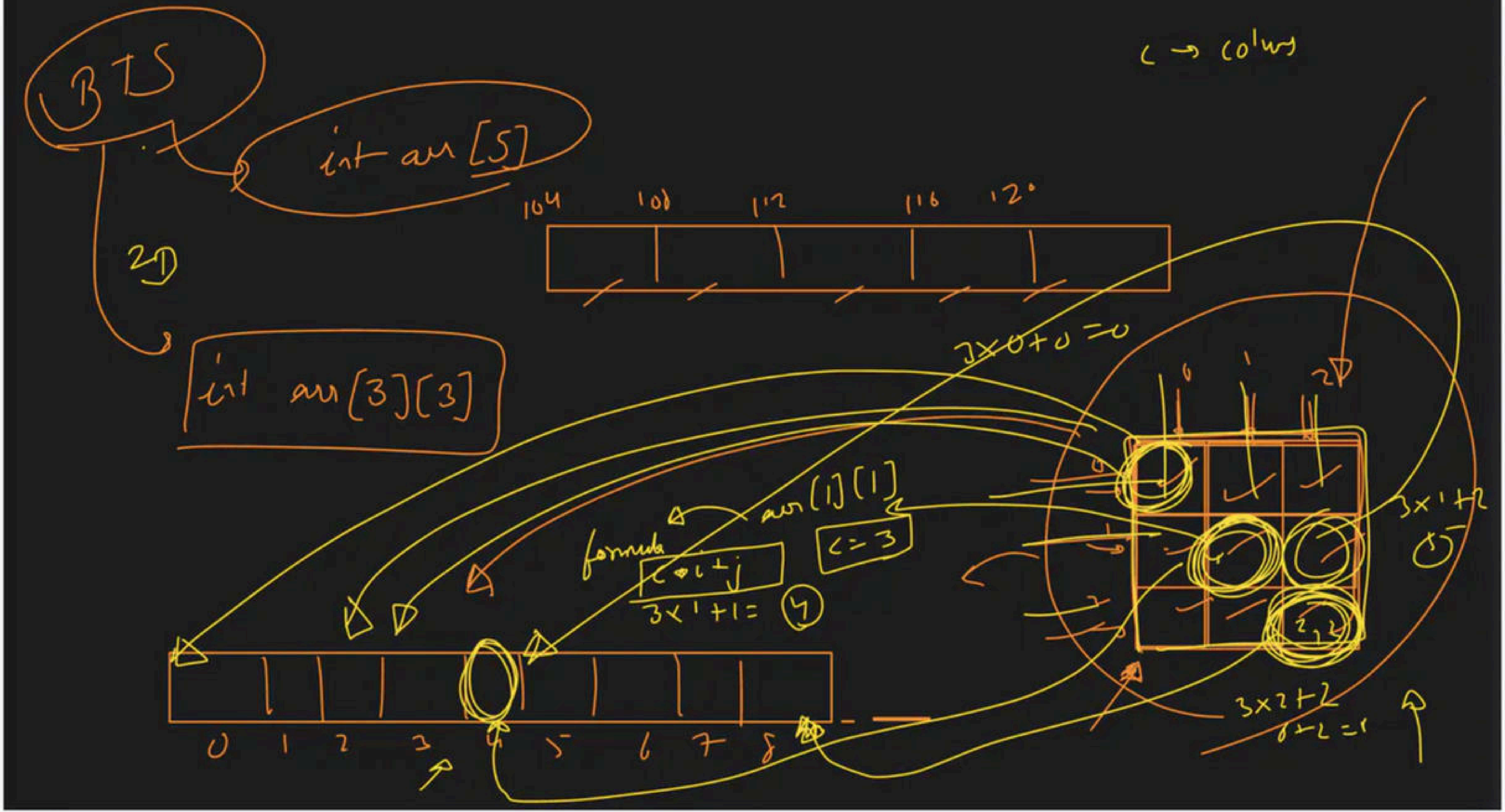
arr[1][1] = 50

arr[1][2] = 60

arr[2][0] = 7

arr[1][1] = 50

arr[2][2] = 90



BTS
int arr[5]
104
108
112
116
120
2D
int arr[3][3]
3×0+0=0
arr[1][1]
formula
c=3
3×1+1= 4
3×1+2
5
3×2+2
0 1 2 3 4 5 6 7 8
2,2

news → Xtra class → 9pm

tomorrow

rows → 5 → r

cols → 4 → c

(1, 2)

$c \times i + j$

$4 \times 1 + 2$

$= 6$

$4 \times 1 + 1$

$= 5$

| | 0 | 1 | 2 | 3 |
|---|---|---|---|---|
| 0 | 10 | 20 | 30 | 40 |
| 1 | 50 | 60 | 70 | 80 |
| 2 | 90 | 100 | 110 | 120 |
| 3 | 130 | 140 | 150 | 160 |
| 4 | 170 | 180 | 190 | 200 |

1,1

1,2

4,2

$c \times i + j$

$4 \times 4 + 2 = 18$

| | | | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | | | 60 | 70 | | | | | | | | | | | | 190 | |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 |

target → row wise access

outer

```
for (int i=0; i<row; i++)
{
    for (j=0; j<col; j++)
    {
        cout << arr[i][j]
    }
}
```

| | 0 | 1 | 2 |
|---|---|---|---|
| 0 | (0,0) 10 | (0,1) 20 | (0,2) 30 |
| 1 | (1,0) 40 | (1,1) 50 | (1,2) 60 |
| 2 | (2,0) 11 | (2,1) 21 | (2,2) 31 |
| 3 | (3,0) 41 | (3,1) 51 | (3,2) 61 |

i=3

i=2
j=0 → 2,0
j=1 → 2,1
j=2 → 2,2

i=1
j=0 → 1,0
j=1 → 1,1
j=2 → 1,2

i=0
j=0 → 0,0
j=1 → 0,1
j=2 → 0,2

→ Column wise access

Outer loop
↳ Column

O.L

```
for (int i=0; i<col; i++)
{
  for(int j=0; j<row; j++)
  {
    cout << arr[j][i]
  }
}
```

Inner loop → row

0
1
2
3

| | 0 | 1 | 2 | 3 |
|---|---|---|---|---|
| | (0,0) | (0,1) 2 | (0,2) 3 | (0,3) 4 |
| | (1,0) 10 | (1,1) 20 | (1,2) 32 | (1,3) 50 |
| | (2,0) 11 | (2,1) 21 | (2,2) 13 | (2,3) 14 |
| | (3,0) 20 | (3,1) 22 | (3,2) 53 | (3,3) 47 |

[ ][ ]

i=0
↳ column index

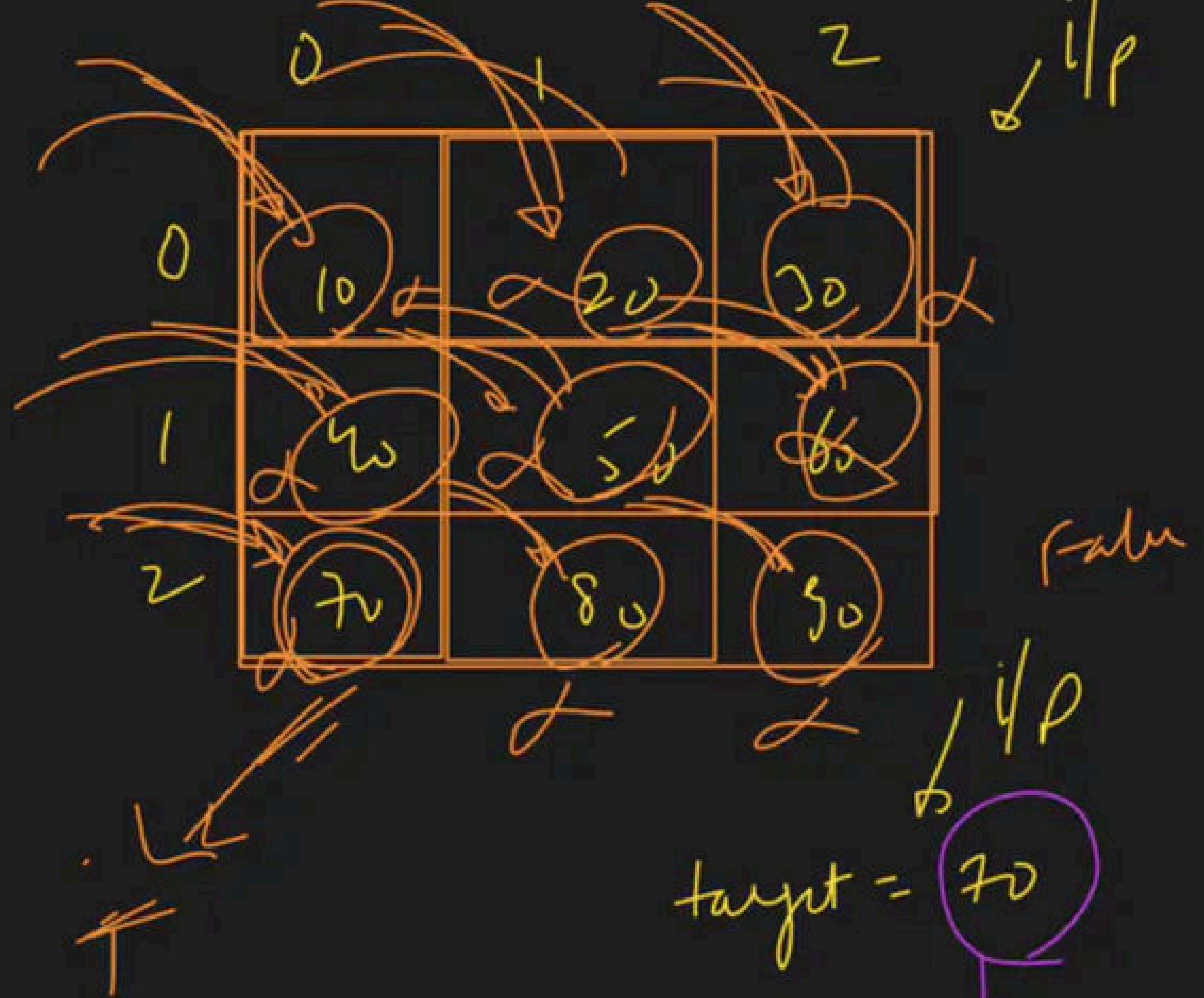
j=0
j=1
j=2
j=3
↑
row index

j=0, j=3

arr[j][i]

→ Searching

o/p → T/F



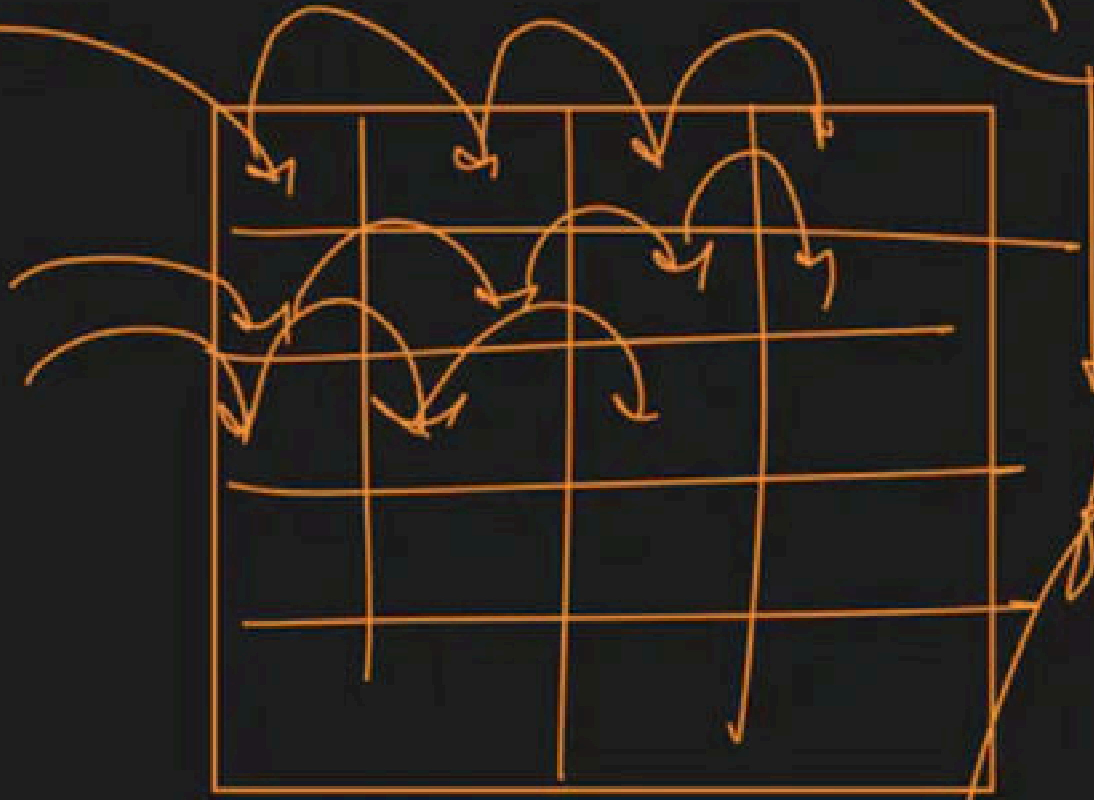

i/p
target = 70

absent
or
present

Max no in an Array

for( )
{
if
}

int maxAns = INT_MIN;



for( )
for( )
{
}

no > maxAns

maxAns = no

## Row wise sum

sum = 0

| row \ col | 0 | 1 | 2 | 3 | |
|---|---|---|---|---|---|
| 0 | 10 | 20 | 5 | 7 | Sum → 42 |
| 1 | 2 | 4 | 6 | 8 | Sum → 20 |
| 2 | 10 | 15 | 15 | 10 | Sum → 50 |

```
for (i → i<r)
{ int sum=0
  for(j → j<c)
  { sum = sum + arr[i][j]
  }
  cout << sum;
}
```

slides → setting → slide mode → download

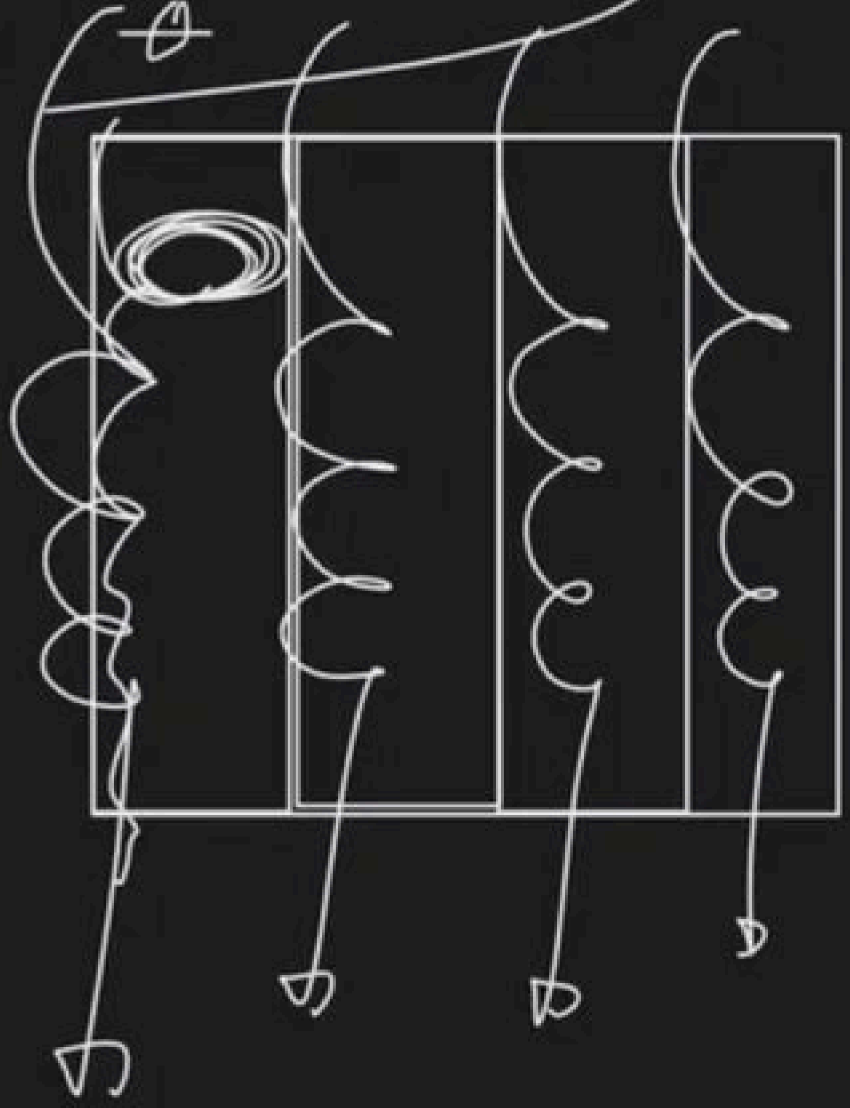
Video lag

Audio lag

Col wise sum

→ col wi

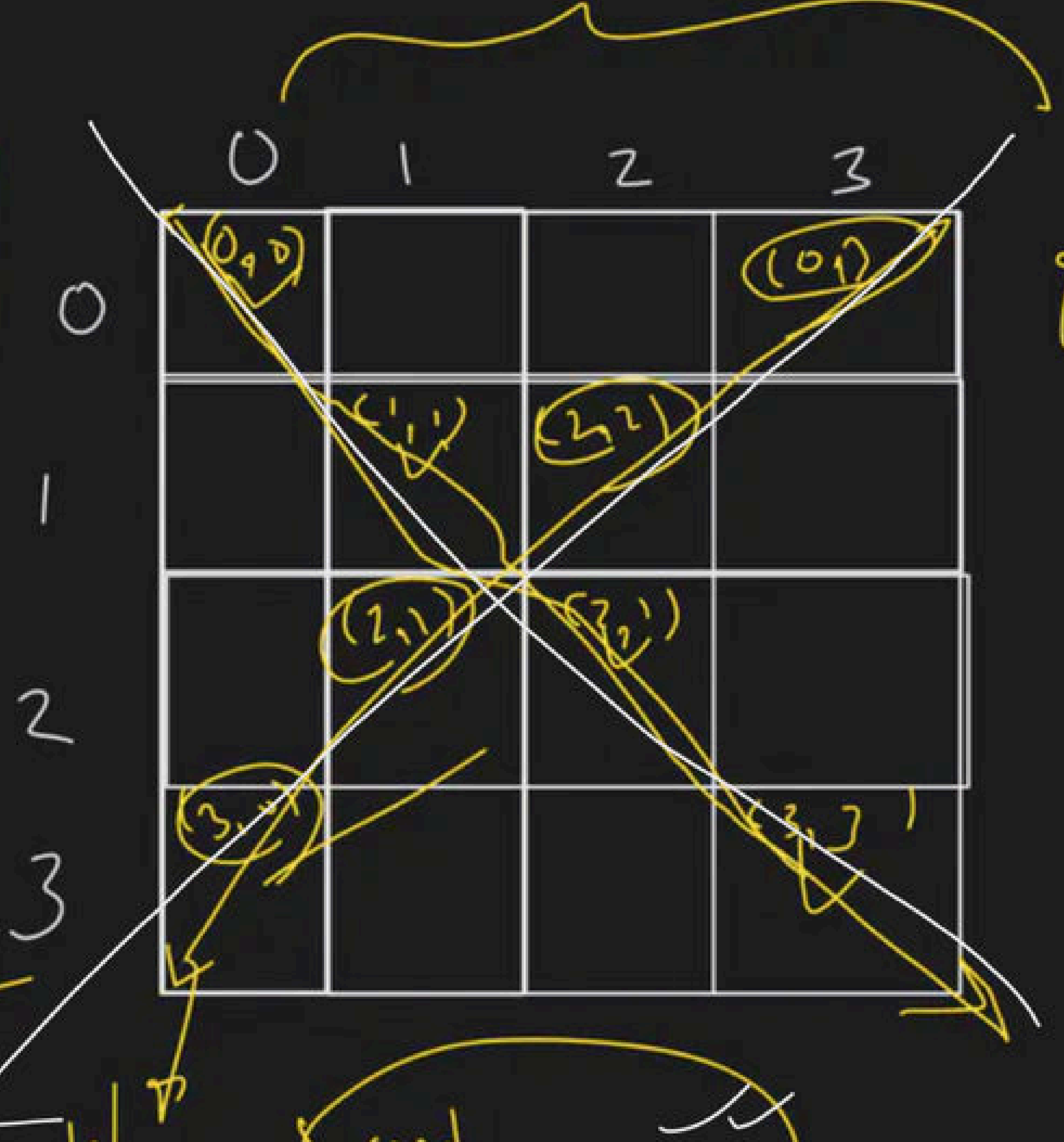
→ col wise sum



```
for (i → i<col )
{ int sum = 0
  for (j → j< row)
  {
    sum = sum +
       arr[j][i]
  } cout << sum;
}
```

Diagonal Print

```
for(i=0; i<row; i++)
{
cout << arr[i][i];
}
```




H/W -> second Diagonal print

```
sum = 0
for(i=0; i<row; i++)
{
sum = sum + arr[i][i];
}
cout << sum;
```

Diagonal sum

| rows ↓ \ col. → | 0 | 1 | 2 |
|---|---|---|---|
| 0 | (0,0) | (0,1) | (0,2) |
| 1 | (1,0) | (1,1) | (1,2) |
| 2 | (2,0) | (2,1) | (2,2) |

$R = 3$

$C = 3$

(0,2)

(1,1)

(2,0)

$i = 0,\ j = 2$

$\Rightarrow (0, C-1) \Rightarrow (0, 2)$

(1, 1)

(2, 0)

i++, j--

& j==0

```
for( ; j>=0; j--)
{
cout << a[i++][j]   (j>=0)
3
```

```
i = R-1, j = 0;

while ( i >= 0 )
{
    cout << a[i][j];
    i--;    j++;
}
```

(2, 0)

(1, 1)

(0, 2)

Dry Run

```
int i = 0;
for ( ; i < N; i++)
{

}
```

```
for (int i = 0; i < N; i++)
{

}
```

# Transpose of a Matrix

arr[0][0] ⟷ arr[0][0]
arr[1][0] ⟷ arr[0][1]
arr[2][0] ⟷ arr[0][2]
arr[2][1] ⟷ arr[1][2]

o/p

i/p

row

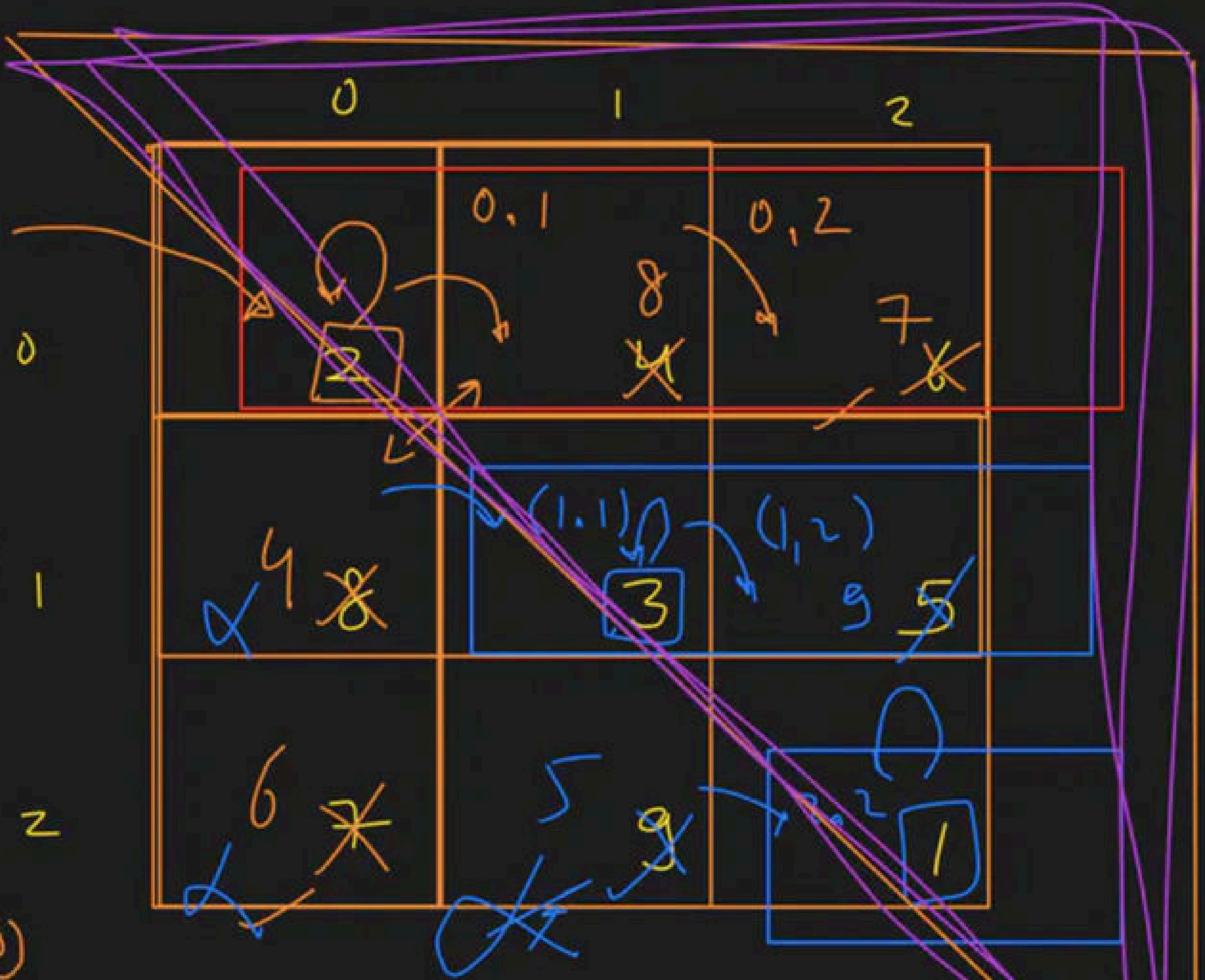
$$\begin{array}{c|ccc} & 0 & 1 & 2 \\ \hline 0 & 2 & 4 & 6 \\ 1 & 8 & 3 & 5 \\ 2 & 7 & 9 & 1 \end{array} \qquad \begin{array}{c|ccc} & 0 & 1 & 2 \\ \hline 0 & 2 & 8 & 7 \\ 1 & 4 & 3 & 9 \\ 2 & 6 & 5 & 1 \end{array}$$

Dry Run

swapping

arr[i][j] ⟷ arr[j][i]

swap(arr[i][j],
arr[j][i])




(0,0)

arr[0][0] ↔ arr[0][0]

(0,1)

arr[0][1] ↔ arr[1][0]

(0,2)

arr[0][2] ↔ arr[2][0]

(1,1)

arr[1][1] ↔ arr[1][1]

(1,2)

arr[1][2] ↔ arr[2][1]

(2,2)

arr[2][2] ↔ arr[2][2]

→ for loop

| | | |
|---|---|---|
| 2 | 8 | 7 |
| 4 | 3 | 9 |
| 6 | 5 | 1 |

```
for (i=0; i<row; i++)

    for (j=0; j<col; j++
```

```
for (i=0; i<row; i++)
{
    for (j=i; j<col; j++
    {
    }
```


Logic swap(arr[i][j] → arr[j][i])

i=0, j=0

swap

(0,1) ⟷ (1,0)

(0,2) ⟷ (2,0)

(0,3) ⟷ (3,0)

(1,0) ⟷ (0,1) ≠

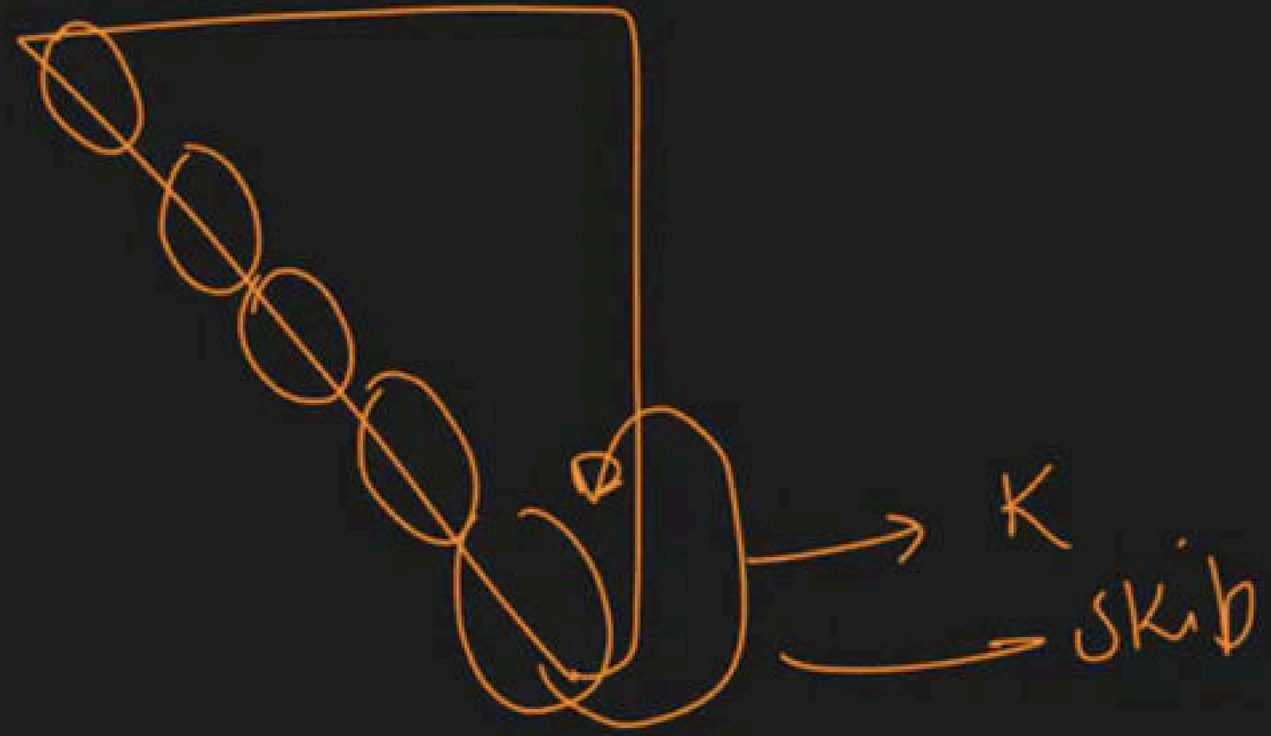
```
for(i=0 → i<row)
{
  for(j=0; j<=i;
  {

  }
}
```

Doubt

J == i ?

J = i ✓

J = i+1



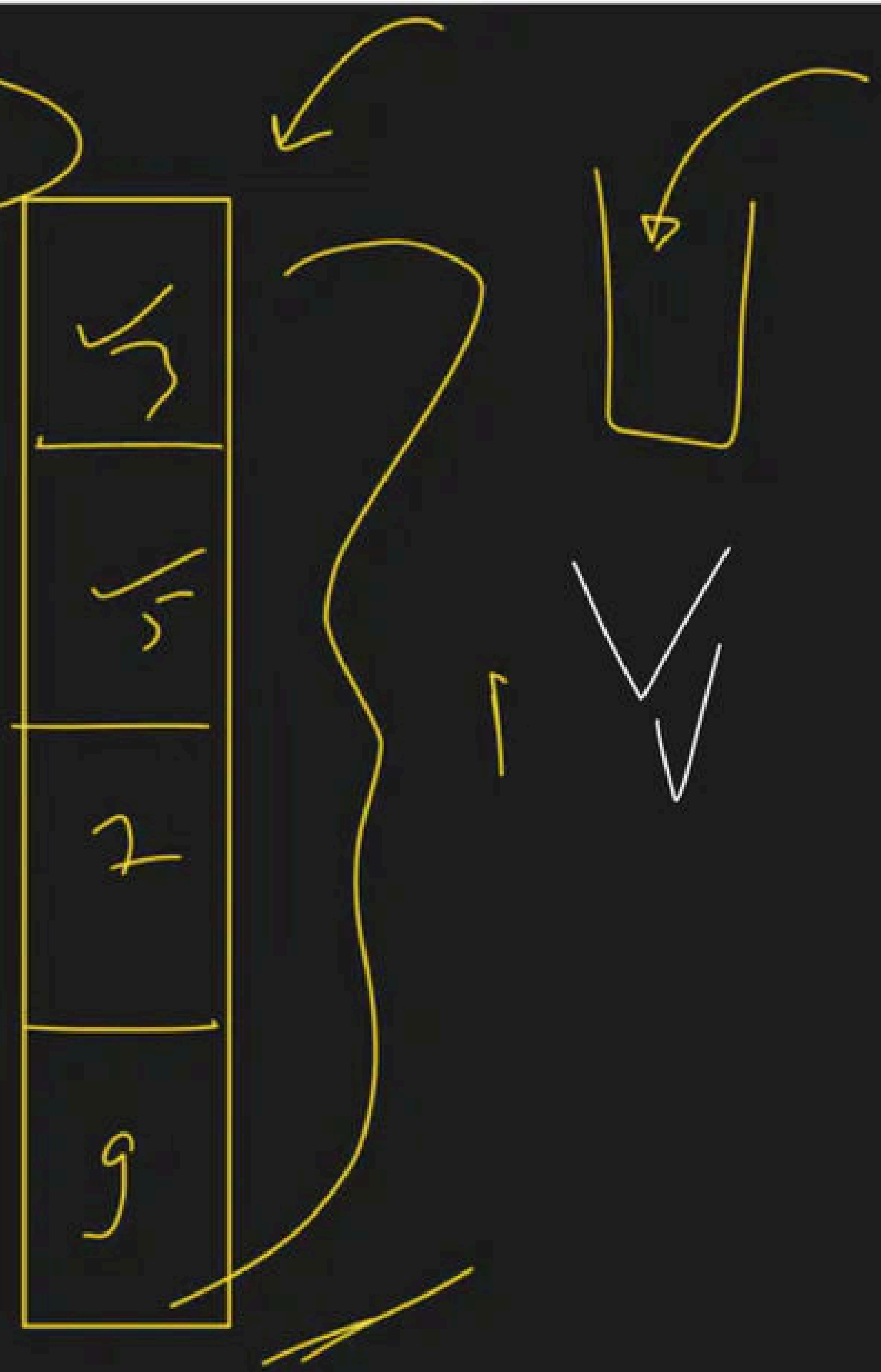
Vector → 2D

1D → vector <int> arr

2D

→ vector <vector<int> > arr

vector<int> v

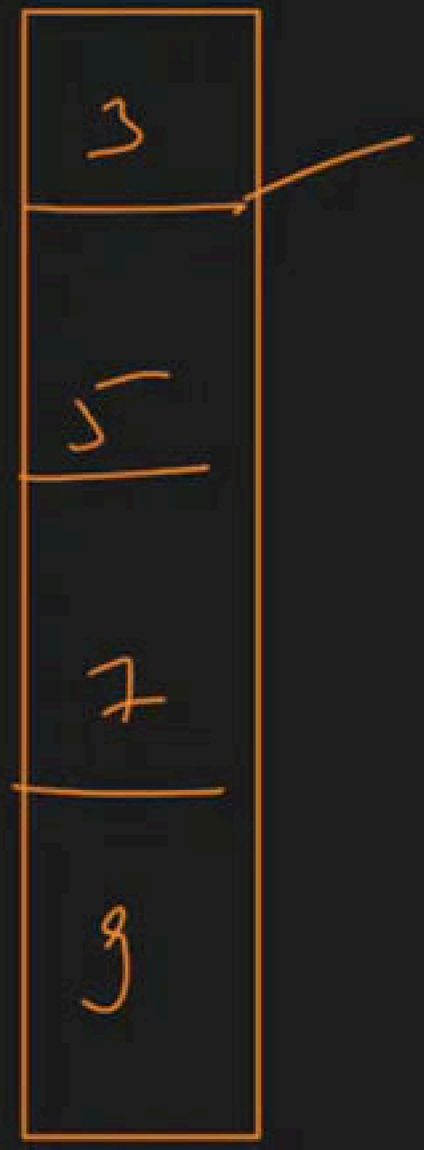


```
vector <int> v
```

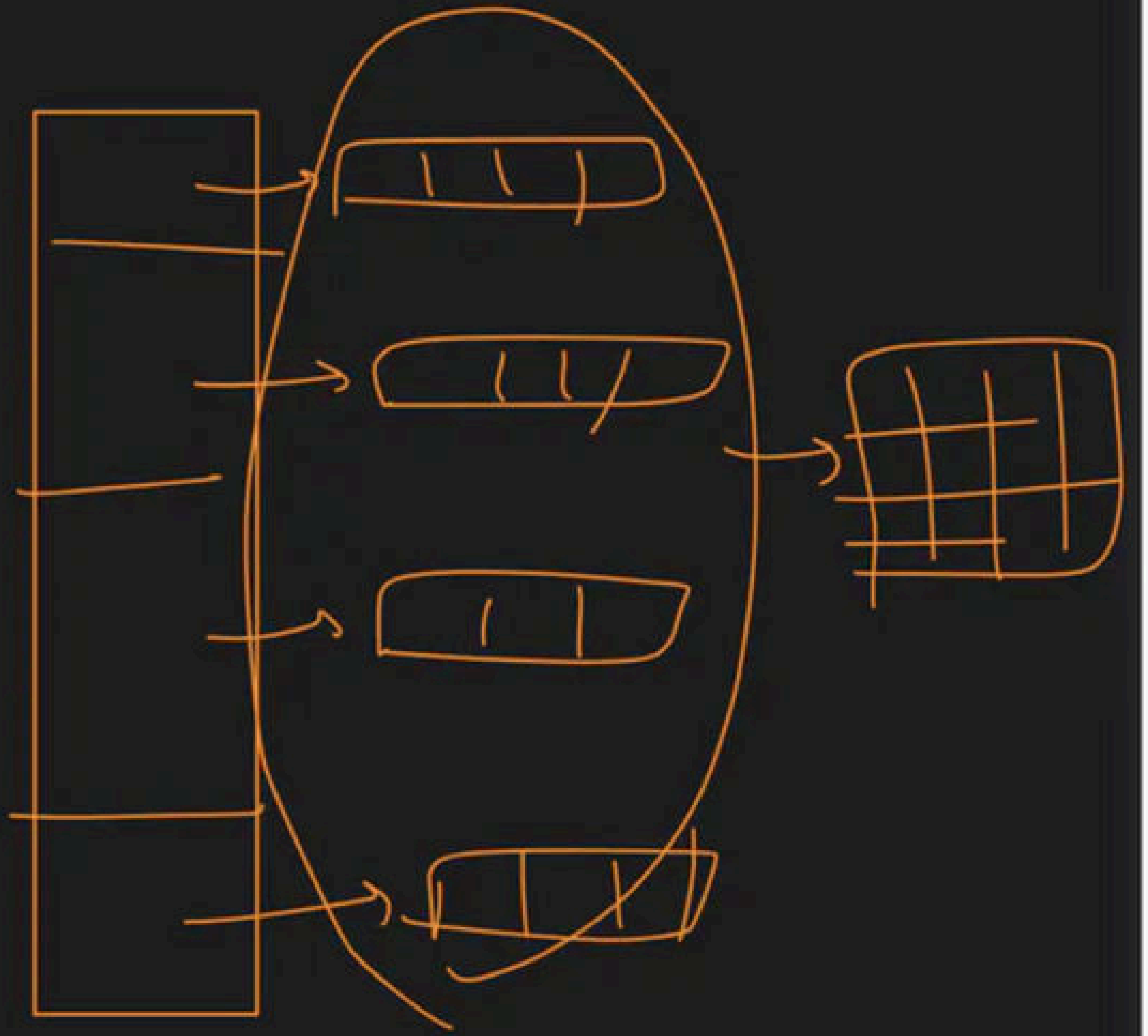
10

| | | | | |
|---|---|---|---|---|
| 3 | 5 | 5 | 7 | 9 |

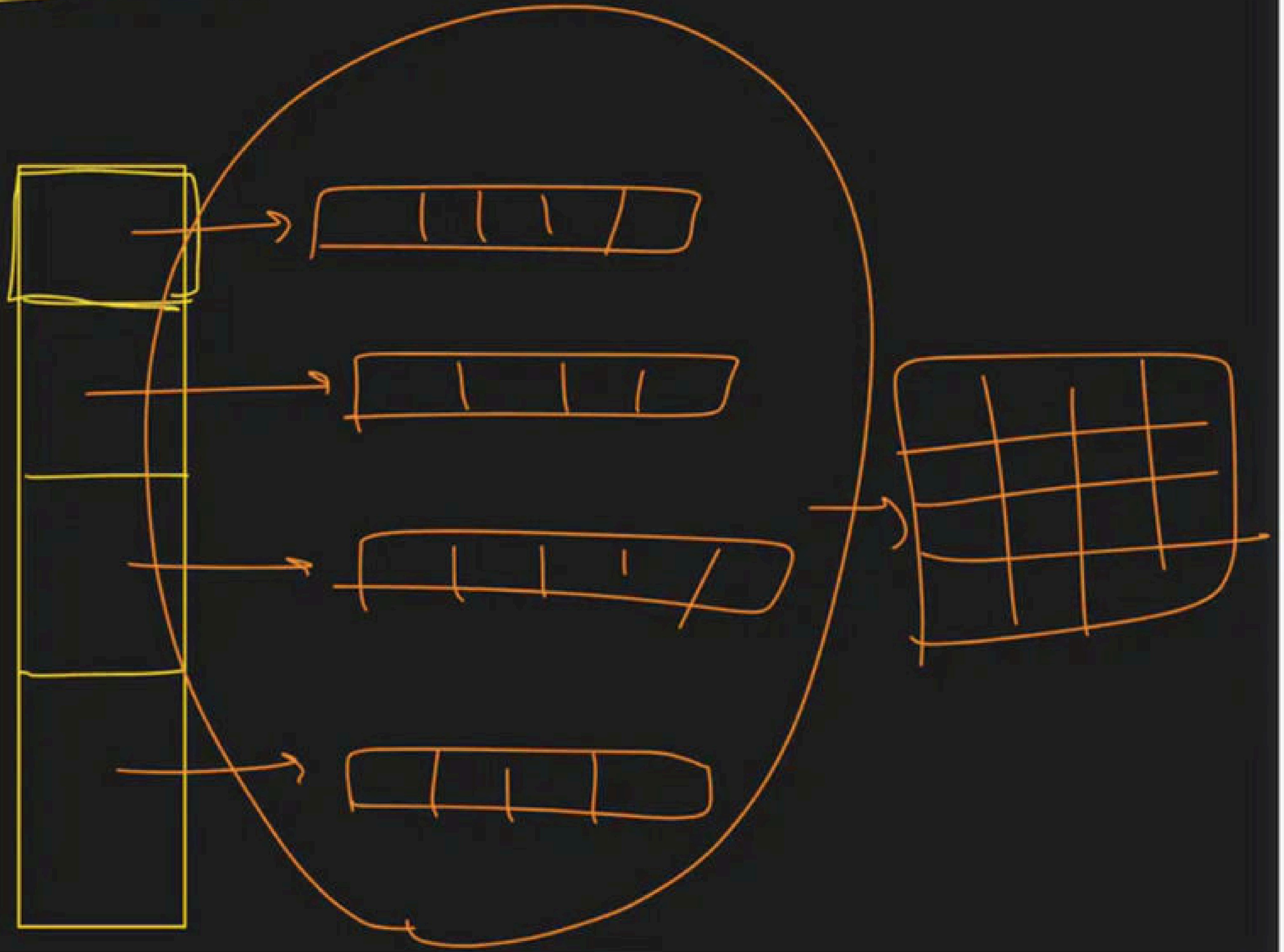
Vector <int>



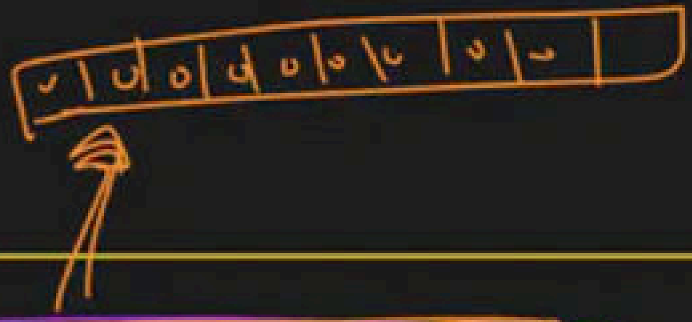

Vector <vector <int>>



vector <vector < int > >



vector <vector <int> > arr



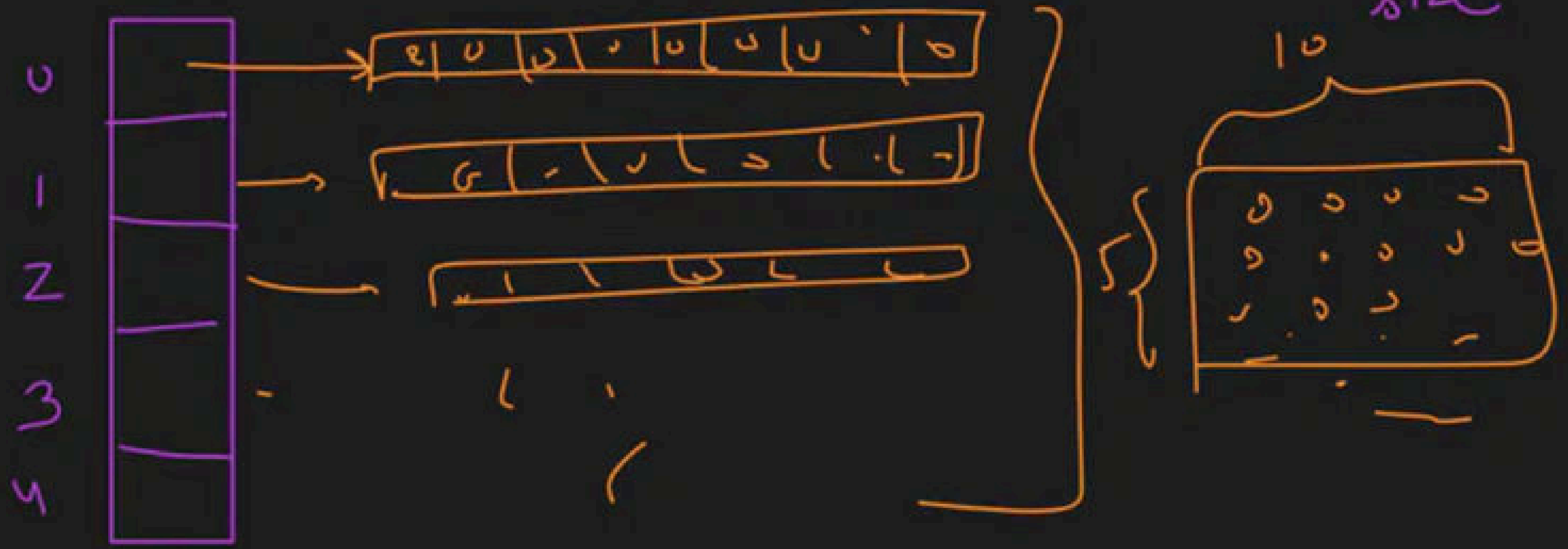
vector <vector <int> > arr (5, vector<int>(10, 0))

2D array

name

row size

row item
↳ initialize
↳ with a vector of size 10 that is initialize with 0



⇒ Swap implemet → (a, b) → Swap.

1 Temp. variable → ✓

2 (+, −) → ~~A~~ Arithematic

3 XOR

(2) (+, -) wala

int a, int b

$a = a + b$

$b = a - b$

$\downarrow$

$5 - 3$

$b = 2$

$a = a - b$

$\downarrow \quad \downarrow$

$5 \quad 2$

$\cancel{2}, \cancel{5}\ 3$ a

$\cancel{3}\ 2$ b

(1) $a = a + b$

(2) $b = a - b$

(3) $a = a - b$

$+ \longrightarrow$ Mehnat

$- \longrightarrow$ mehnat

$\Rightarrow$ XOR

NOT, OR, AND, XOR $\longrightarrow$ Bitwise $\longrightarrow$ method --

int a , int b

$\downarrow$ 2 $\qquad \downarrow$ 3

$a = a \wedge b = \boxed{2 \wedge 3}$

$b = 2 \wedge \cancel{3} \wedge \cancel{3} \Rightarrow 2$

$a = 2 \wedge 3 \wedge 2 \Rightarrow 3$

① $a = a \wedge b$

② $b = a \wedge b$

③ $a = a \wedge b$

# Arrays - [Extra Class]

Special class

Love Babbar • Sept 12, 2023

→ i/p → [ ] → {23, -7, 12, -10, -1, 40, 60}

modify -> -ve → left

+ve → right

| | | | | | | |
|---|---|---|---|---|---|---|
| -7 | -10 | -11 | 23 | 12 | 40 | 60 |

$\boxed{\text{2 min}}$

$\{\underbrace{-7, -10, -11}_{-ve}, \underbrace{23, 12, 40, 60}_{+ve}\}$

Approach:-

(A) Sorting → O(nlogn) X
greedy

(B) 0 & 1 wala logic → $\boxed{\text{"Sort 0's \& 1's"}}$ → Counting X

→ 2 pointer approach — O(n)

(C) temp array ← O(n) ← T.C

S.C → $\boxed{\text{O(n) space}}$

index → traverse array

arr

| 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| -7 | -10 | -11 | 23 | 12 | 40 | 60 |

index index index index index index index index

j j j j

j → -ve element ko rakh skte hai

arr(index) > 0 → ignore

arr[index] < 0 → swap(arr(index], arr[j]) → j++

~~arr(index) = 23~~

arr[index] = -7

~~arr(index) = 12~~

arr[index] = -10 → swap

arr(index) = -11 → swap

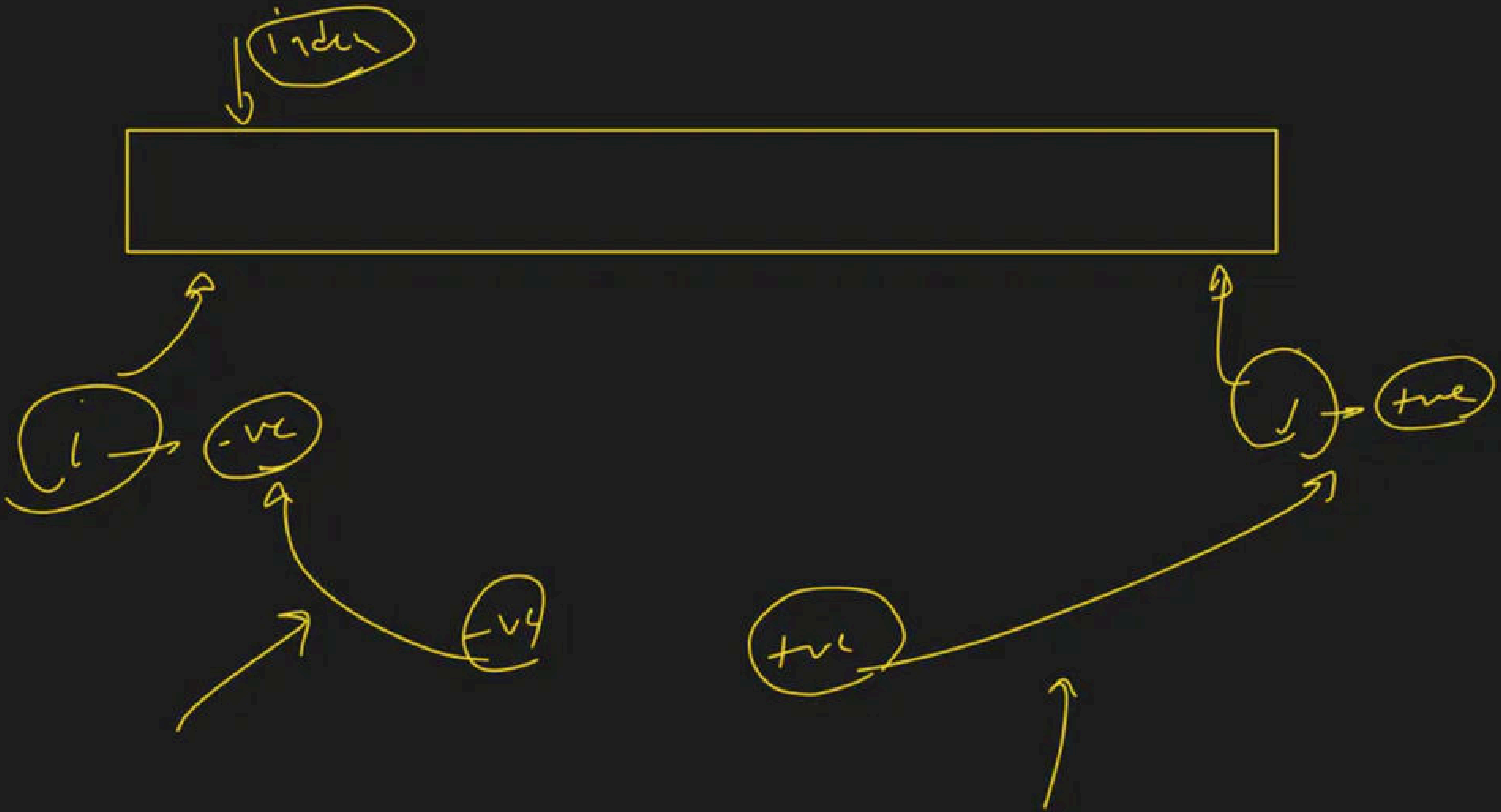
```
for (index = 0; index < n; index++)
{
    if (arr[index] < 0)
    {
        swap(arr[index], arr[j])
        j++;
    }
}
```

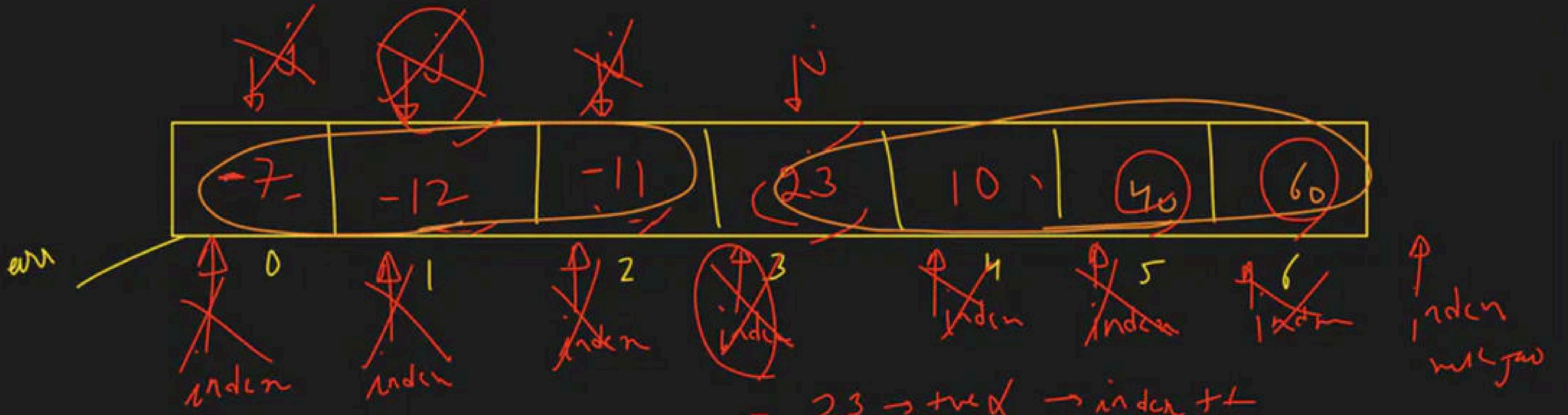
O(n)

$l = 0$

$h = n-1$

```
int index = 0;
while(index < n)
{

    index++;
}
```


index
i
-ve
-ve
true
j
true




23 → +ve → index++

-7 → -ve → swap → j++ → index++

10 → +ve → index++

-12 → -ve → swap → j++ → index++

-11 → -ve → swap → j++ → index++

40 → +ve → index++

60 → +ve → index++

→ Sort 0's 1's & 2's ✓

zerocount = 2
onecount = 3
twocount = 2

i/p →

| | | | | | | |
|---|---|---|---|---|---|---|
| 0 | 1 | 1 | 2 | 0 | 2 | 1 |

arr

o/p →

| | | | | | | |
|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 1 | 2 | 2 |

0's one 2's

**Approach**

- → counting → $O(n)$ ✓
- → sorting → $O(n \log n)$ ✓
- → 2 pointer approach

~~index~~ → array ~~data~~ traversal ← index

| 0 | 1 | 0 | 2 | 1 | 0 | 1 | 2 |
|---|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |

arr

left 0/1

0 | 1 | 2

right

left ← index → jahan par nahi "0" rakh sakte

right → index jahan par nahi "2" rakh sakte h

**Logic**

0 mila to left ko dedunga

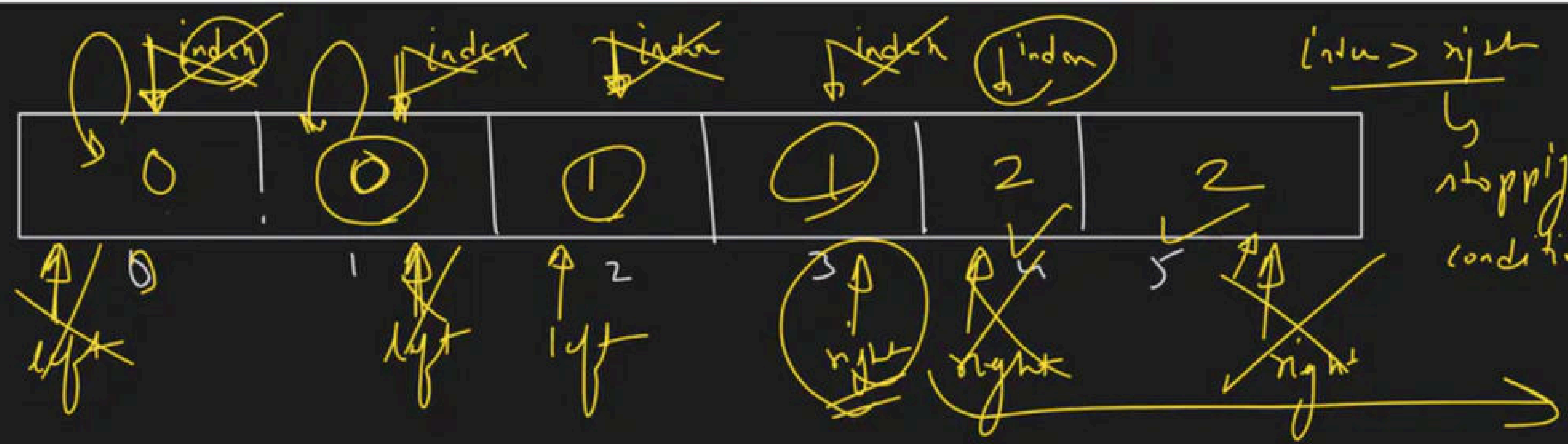
2 mila toh right ko dedunga

1 mila → ignore

→ arr(index) = 1 → X → index++

→ arr[index] = 0 → swap → arr(index), arr(left) → left++ → index++

→ arr(index) = 2 → swap → arr(index), arr(right) → right-- → index++




index > right → stopping condition

arr[index] = 2 → swap → index, right → right-- →

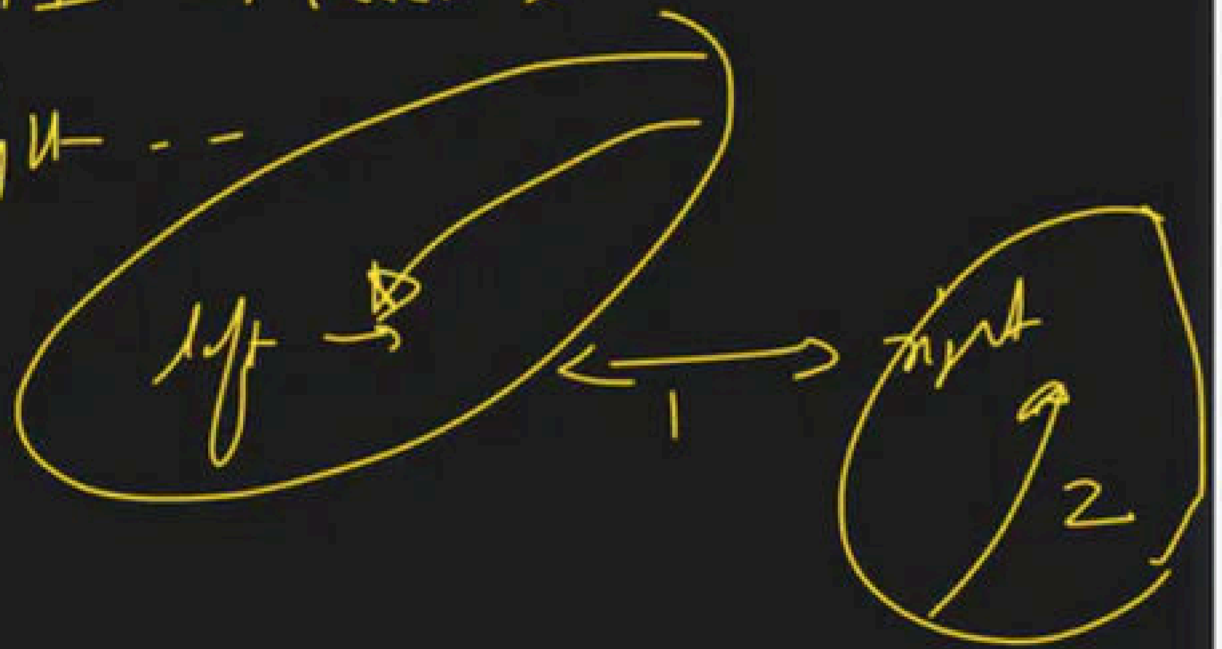
arr[index] = 0 → swap → index, left → left+1 → index+1

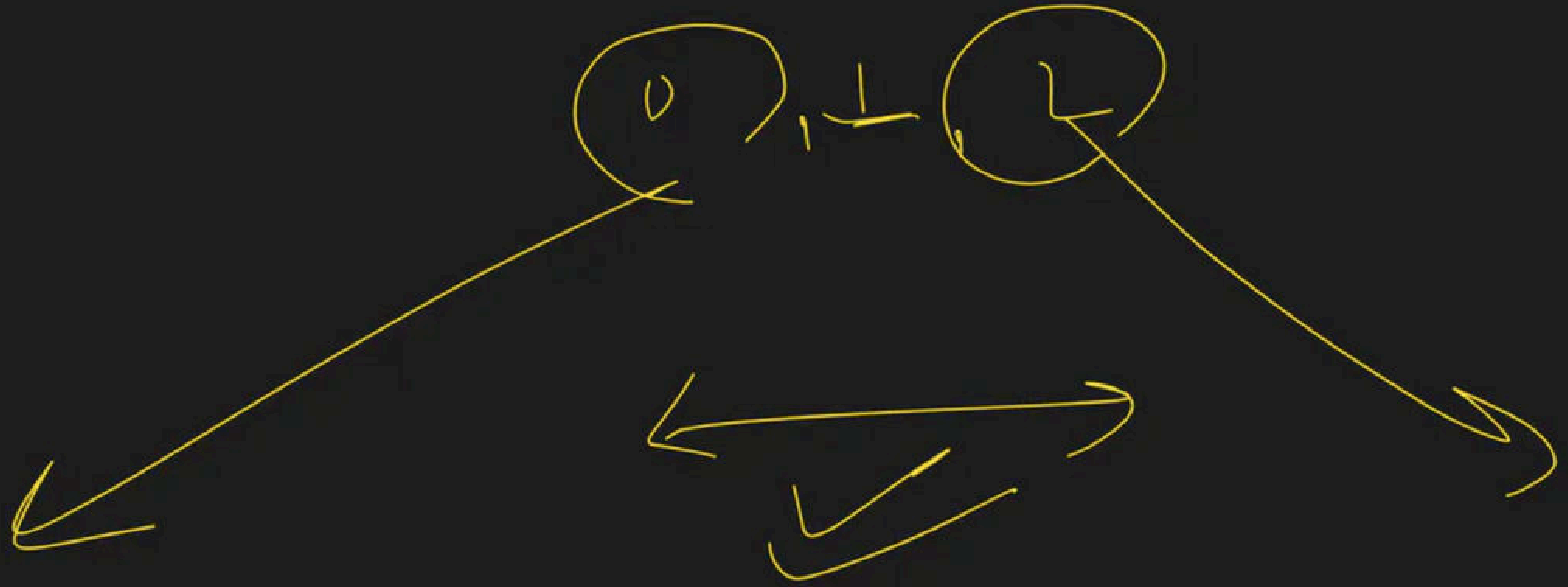
arr[index] = 0 → swap → index, left → left+1 → index+1

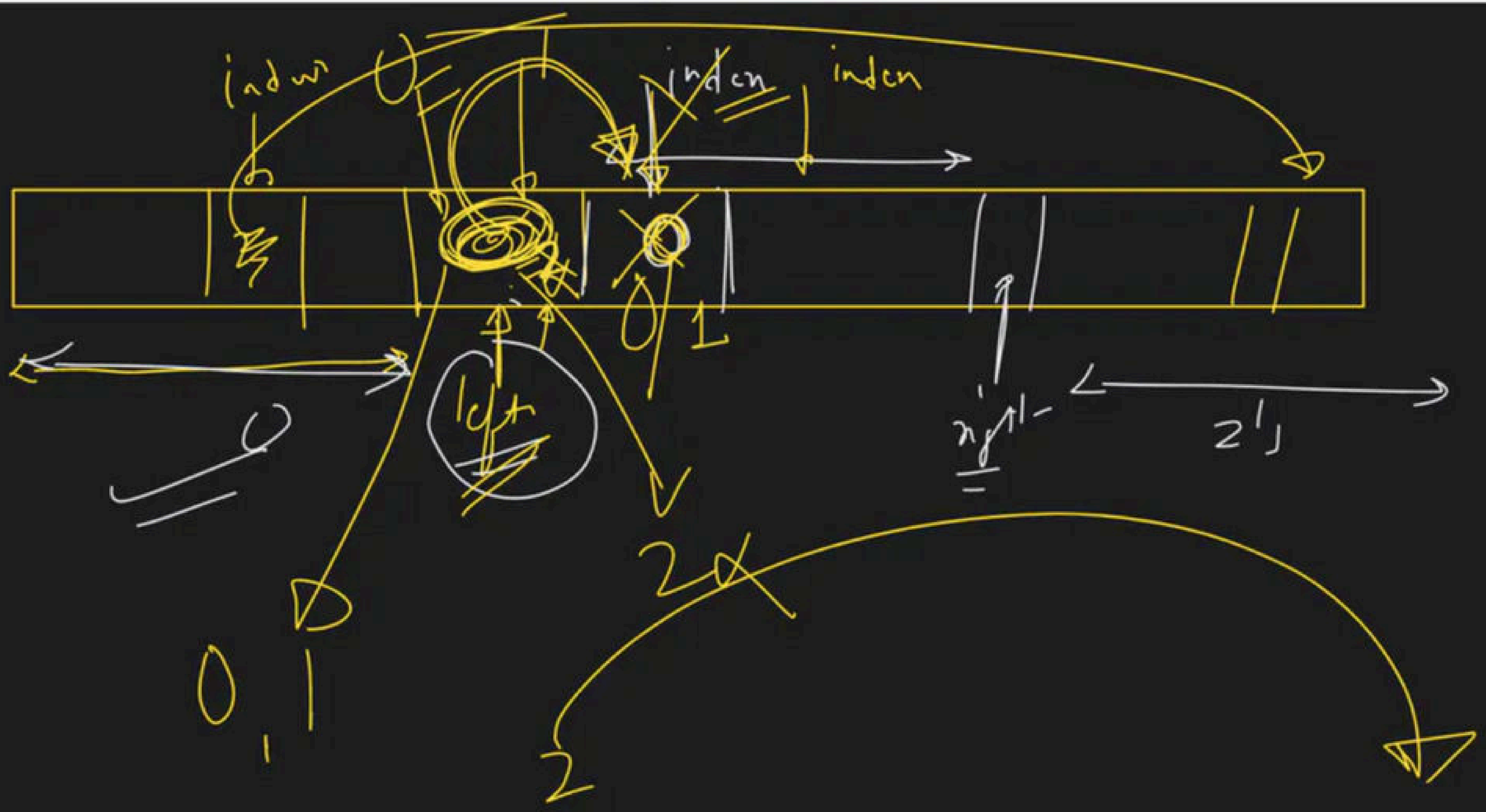
arr[index] = 2 → swap → index, right → right--

arr[index] = 1 → index+1

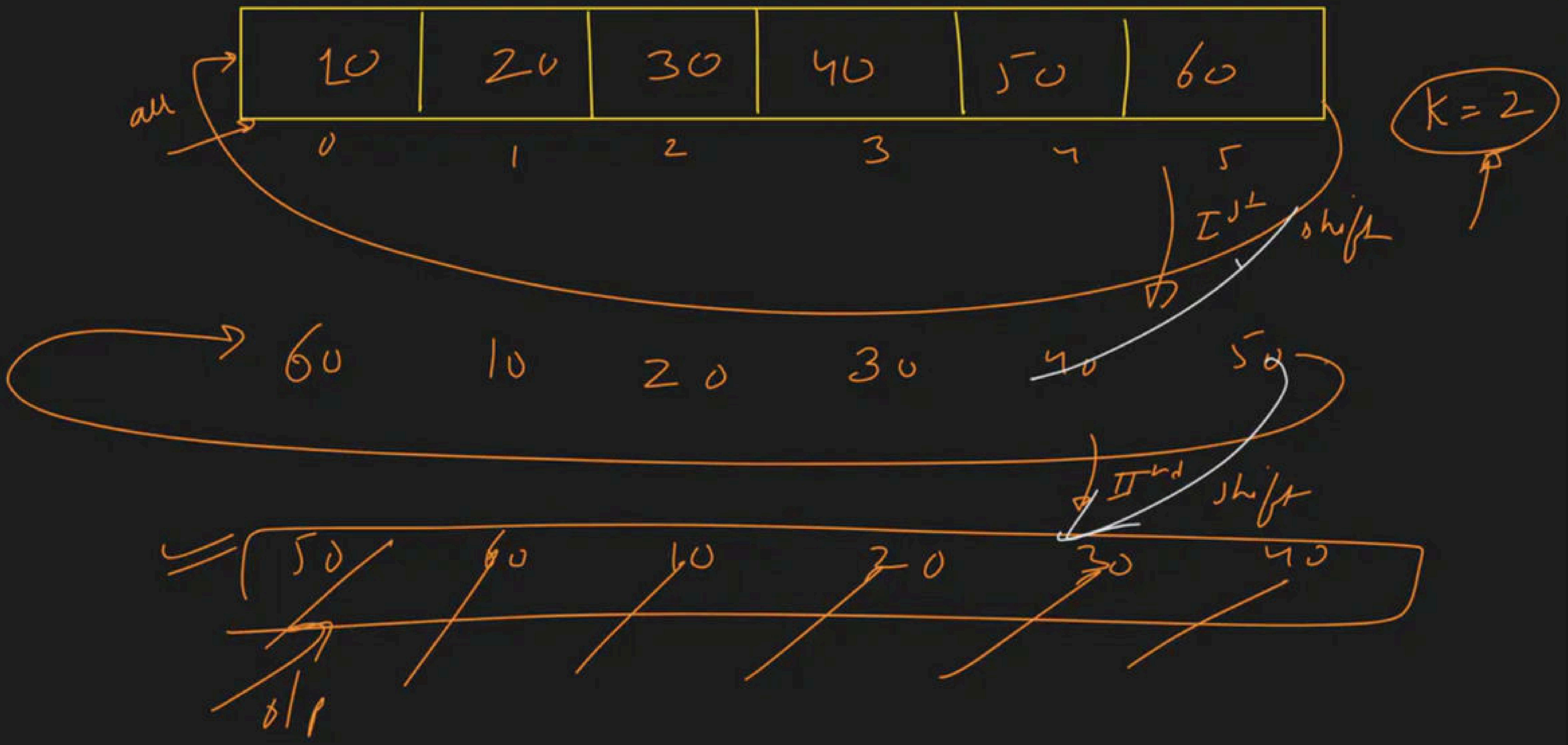
arr[index] = 1 → index++




0
1
2



indwn
inden
inden
0
1
0
left
0, 1
2
2


arr
10 20 30 40 50 60
0 1 2 3 4 5
k = 2
I^st shift
60 10 20 30 40 50
II^nd shift
= 50 60 10 20 30 40
o/p

$(0+2)\%6 \to 2\%6 \to 2$

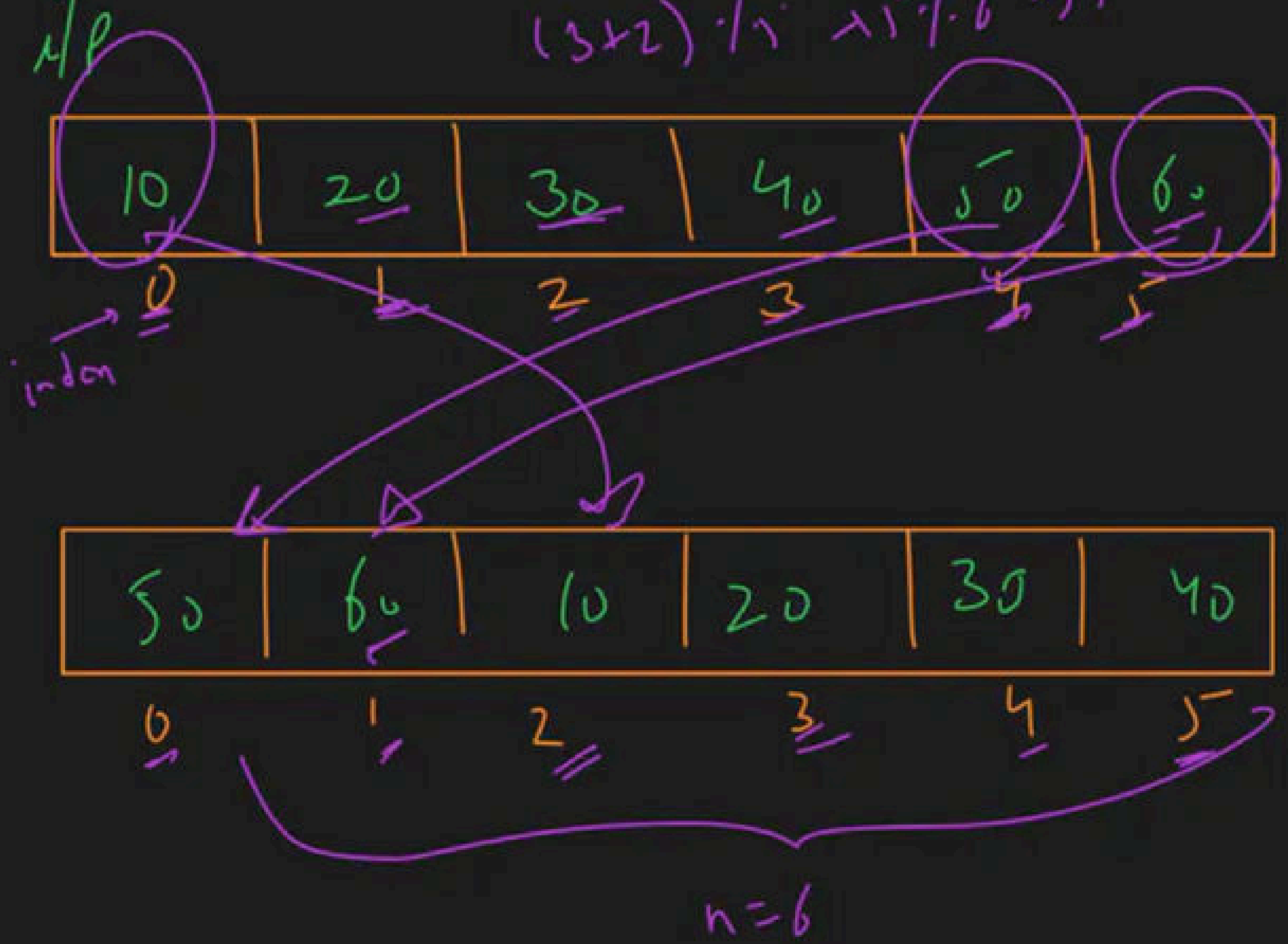
$(1+2)\%6 \to 3\%6 \to 3$

$(2+2)\%6 \to 4\%6 \to 4$

$(3+2)\%6 \to 5\%6 \to 5$

$(4+2)\%6 \to 6\%6 \to 0$

$(5+2)\%6 \to 7\%6 \to 1$




approach

1. modulo ka use karlenge

2. temp array

| i/p | | o/p |
|---|---|---|
| 0 | $\xrightarrow{+2}$ | 2 |
| 1 | $\xrightarrow{+2}$ | 3 |
| 2 | $\xrightarrow{+2}$ | 4 |
| 3 | $\xrightarrow{+2}$ | 5 |
| 4 | $\xrightarrow{+2}$ | 0 |
| 5 | $\xrightarrow{+2}$ | 1 |

7

index + k

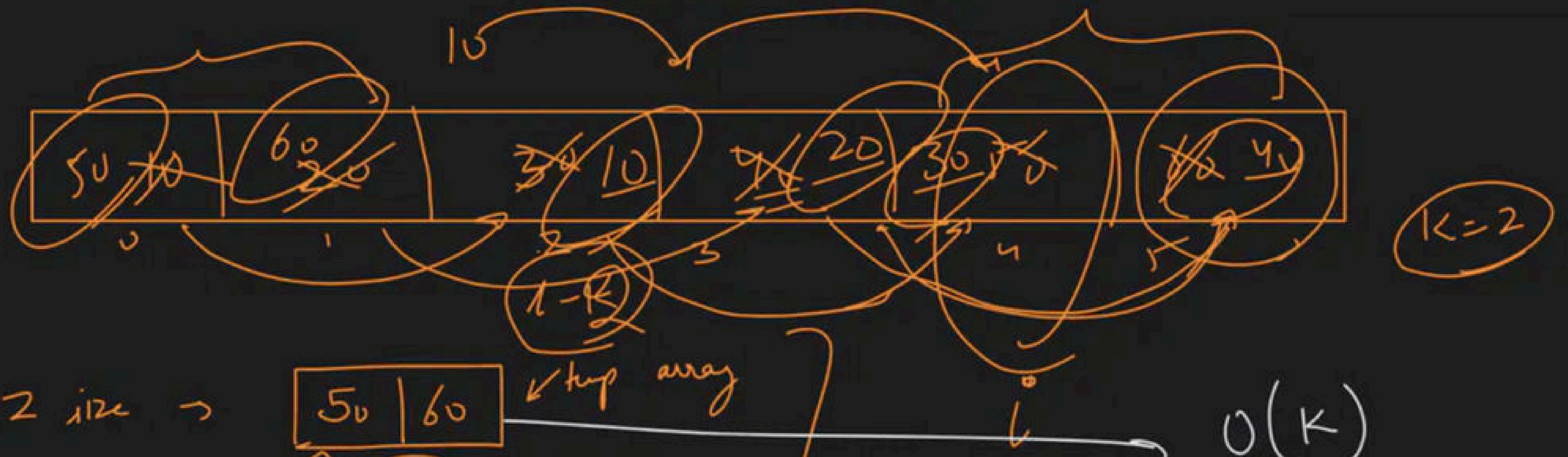
$(\text{index}+k)\%n$

$4+2 \to 6$

index + k - n

$0+2-6$

$2-6 \to \boxed{-4}$




(A) 2 size → 50 | 60 ← tmp array → O(K)

(B) for (n-1 → >=0) → wrong

{

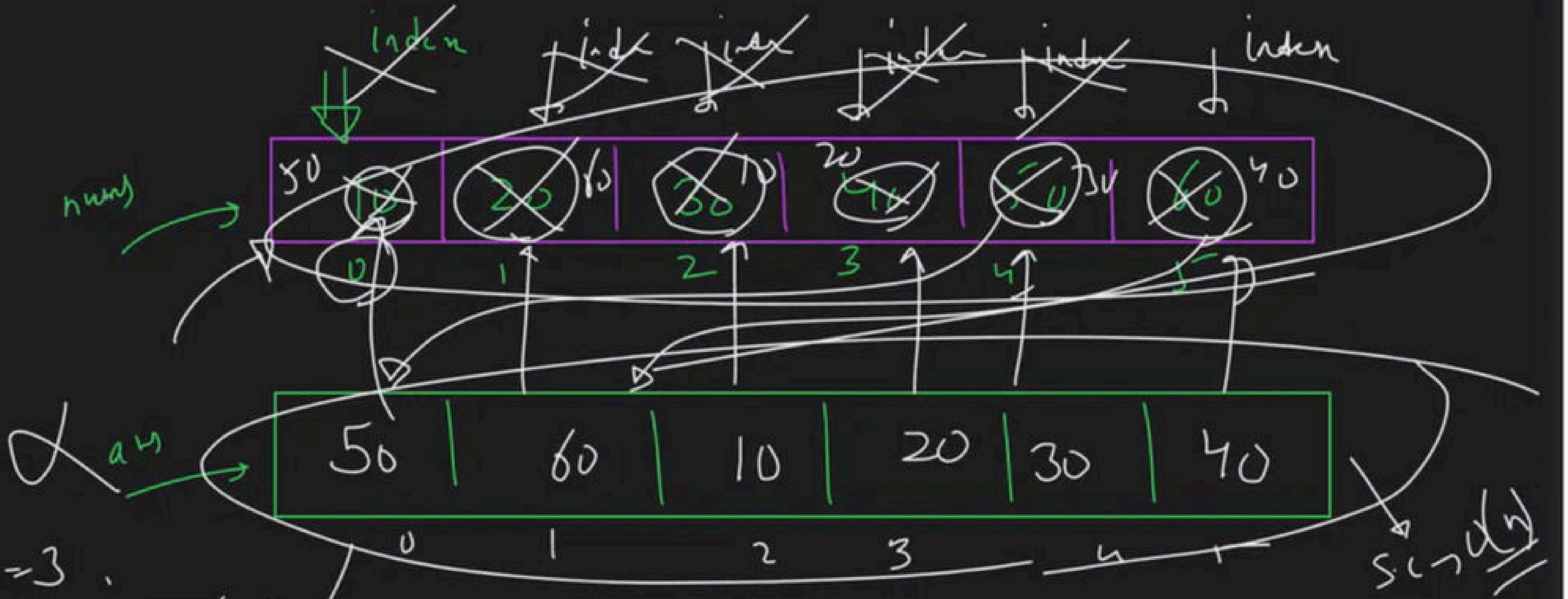
arr[i] = arr[i-K]

}

(C) ~~cop~~ paste → tmp array

O(1) → ?

k = 2
n = 6




index = 3

newIndex = (3+2) % 6 = 5

ans[5] = nums[3]

index = 4

newIndex = (4+2) % 6 = $\boxed{0}$

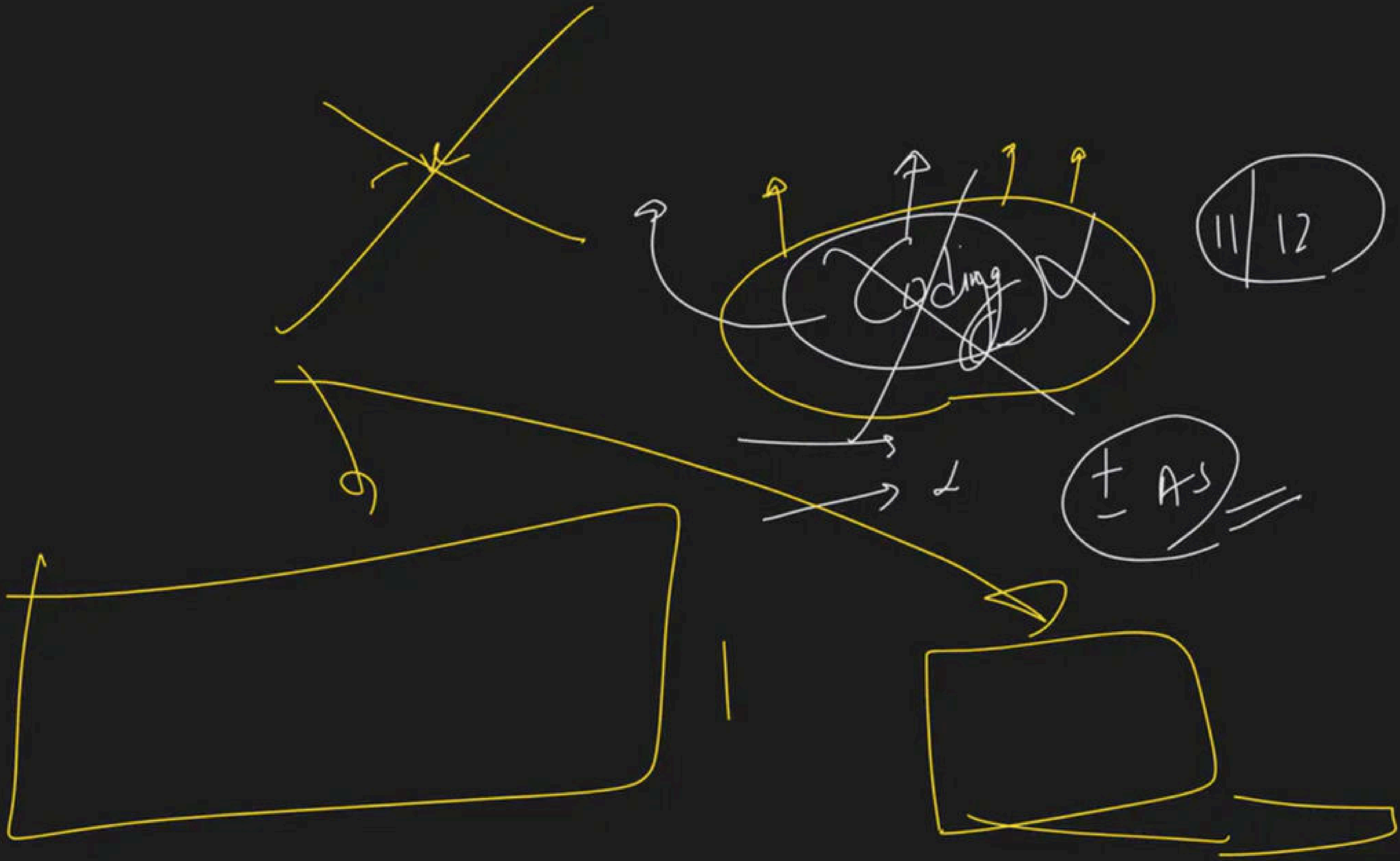
ans[0] = nums[4]

index = 5

newIndex = (5+2) % 6 = 1

ans[1] = nums[5]

nums = ans


Coding
11/12
± AS

Rotate Array → interview Q

1. Extra space → Extra space

2. Single rotate k Times → k Time

$O(n)$ ← 3. Modulus → $(i+k) \% n$

$O(n)$ ← 4. Reversal method.

2)

| 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 |

k = 3

① 

| 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| 7 | 6 | 5 | 4 | 3 | 2 | 1 |

② 

| | | | | | | |
|---|---|---|---|---|---|---|
| 5 | 6 | 7 | 4 | 3 | 2 | 1 |

③ 

| | | | | | | |
|---|---|---|---|---|---|---|
| 5 | 6 | 7 | 1 | 2 | 3 | 4 |

**Algo**

① reverse whole array
Rev(0, n-1)

② Rev(0, k-1)

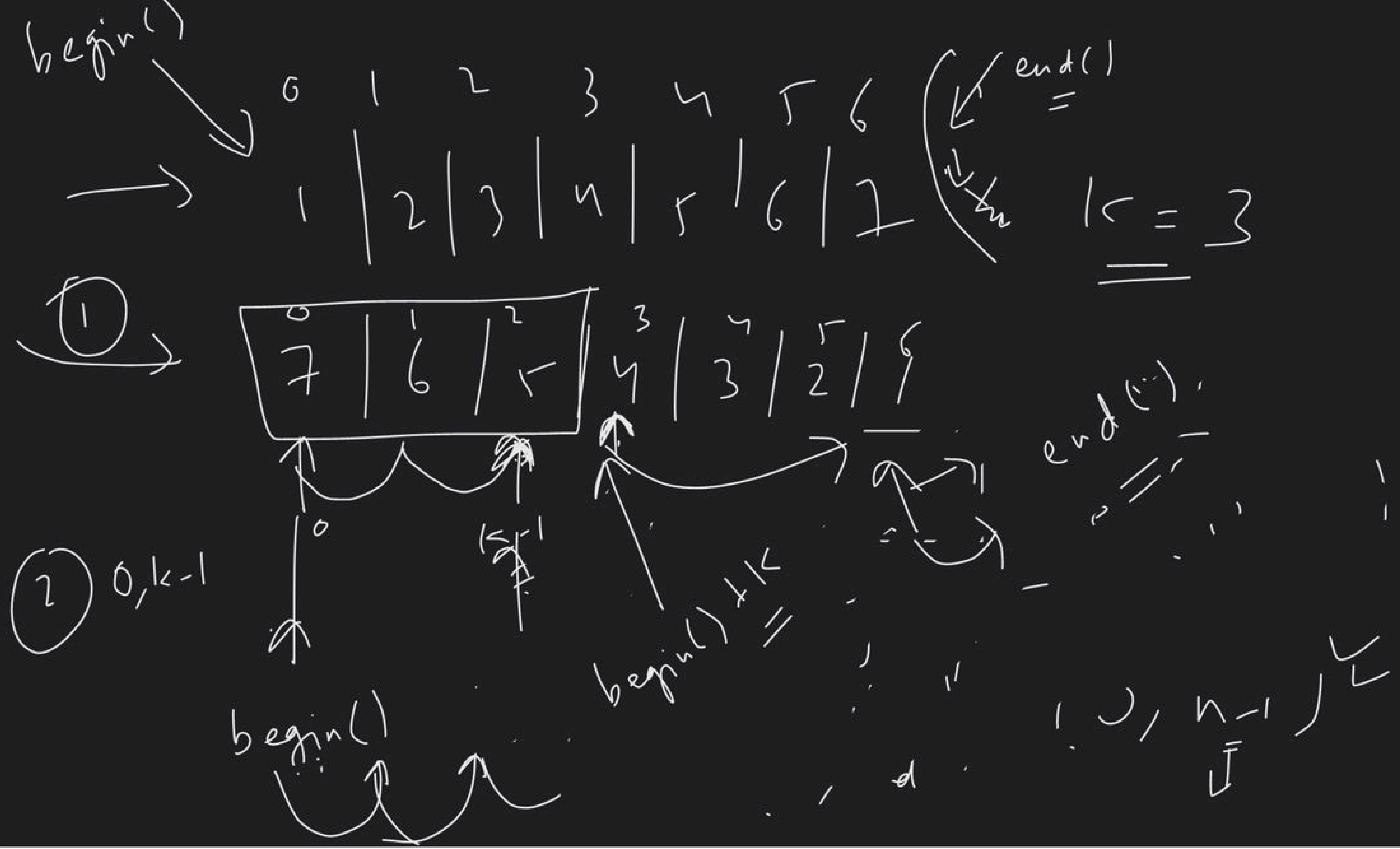
③ Rev(k, n-1)

$$\Rightarrow \quad \begin{array}{c|c|c|c} 0 & 1 & 2 & 3 \\ -1 & -100 & 3 & 99 \end{array} \qquad k = 2$$

1. $RE(0, n-1) \rightarrow \begin{array}{c|c|c|c} 0 & 1 & 2 & 3 \\ 99 & 3 & -100 & -1 \end{array}$

2. $RE(0, k-1) \rightarrow \begin{array}{c|c|c|c} 3 & 99 & -100 & -1 \end{array}$ $\uparrow_k$ $\uparrow_{n-1}$

3. $RE(k, n-1) \rightarrow \begin{array}{|c|c|c|c|} 3 & 99 & -1 & -100 \end{array}$



begin()
0 1 2 3 4 5 6
1 2 3 4 5 6 7
end()
k = 3
1
0 1 2 3 4 5 6
7 6 5 4 3 2 1
end()
2
0, k-1
0
k-1
begin()+k
begin()
(0, n-1)

| 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 |

$K = 0$

$K = 5$

$K = 5$

$\equiv$

$\Rightarrow$

$K = 0$

$K = 5$

$K = 10$

$K = 0 \rightarrow 1\ 2\ 3\ 4\ 5$

$K = 1 \rightarrow 5\ 1\ 2\ 3\ 4$ $\leftrightarrow K = 6 \Rightarrow$

$K = 2 \rightarrow 4\ 5\ 1\ 2\ 3$

$K = 3 \rightarrow 3\ 4\ 5\ 1\ 2$

$K = 4 \rightarrow 2\ 3\ 4\ 5\ 1$

$K = 5 \rightarrow 1\ 2\ 3\ 4\ 5$

$K = K \% N$

| | | | | | | |
|---|---|---|---|---|---|---|
| 4 | 7 | 3 | 2 | 5 | 6 | 8 |

[1 → 8]

missing no
Konsa hai ?

Approach → for ( ( ) ) → number → search → linear scan

→ $O(n^2)$

sum

sort → n log n

→ traverse → □ □ → 1 diff ann

LC → 268

$\Rightarrow$ 

| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|
| 9 | 6 | 4 | 2 | 3 | 5 | 7 | 0 | 1 |

$N = size = 9$

ans = 0,

element XOR ↳ R ∈ $\boxed{[0, n]}$

$\Rightarrow$

ans = $\cancel{9} \wedge \cancel{6} \wedge \cancel{4} \wedge \cancel{2} \wedge \cancel{3} \wedge \cancel{5} \wedge \cancel{7} \wedge \cancel{0} \wedge \cancel{1} \wedge$

$a[i] \in [0, 9]$

$\boxed{A \wedge 0 = A}$

$\cancel{0} \wedge \cancel{1} \wedge \cancel{2} \wedge \cancel{3} \wedge \cancel{4} \wedge \cancel{5} \wedge \cancel{6} \wedge \cancel{7} \wedge 8 \wedge \cancel{9}$ → 0 → 9 XOR's

↳ 8

Sovan's Doubt AA

Sovan's IQ = x

① Note making

Live class → sampho

① Repeat → Verbose → detailed

```
for(i=0; i<n; i++)
{ if(num==a[i])
    retn i;
}
```

yab num

a[i] ———

— — . - - -

a — — —

② daily 2 Qustion at least

New Q's

Go to the notebook

& Revise 2 Ques

with all approaches

~~Sonal~~

Sovan → Github

week 3

code

$1 \rightarrow 8$

i/p A → 

| | | | | | | |
|---|---|---|---|---|---|---|
| 1 | 8 | 3 | 2 | 7 | 5 | 6 |

→ Sum → $\boxed{32}$

B →

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |

→ Sum → 36

$[1] \rightarrow [n]$

missing no

no of terms

last term

A.P → Sum = $\frac{n}{2}(1+n)$ , $\frac{n}{2}(\text{first term} + \text{last term})$

$36 - 32 = 4$

missing no

$\frac{8}{2}(1+8) = 4 \times 9 = 36$

⇒

| 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 3 | 1 | -2 | -5 | 2 | -4 |

even len

⇒

| 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 3 | -2 | 1 | -5 | 2 | -4 |

Ans

M1 ⇒ $N = 6$, Temp size $N/2 = 3$

Pos →

| 0 | 1 | 2 |
|---|---|---|
| 3 | 1 | 2 |

neg →

| 0 | 1 | 2 |
|---|---|---|
| -2 | -5 | -4 |

⇒ ans ⇒

| 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 3 | -2 | | | | |

$i$ ⇒ 0

| 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 3 | 1 | -2 | -5 | 2 | -4 |
| ~~i~~ | ~~i~~ | ~~i~~ | ~~i~~ | ~~i~~ | i |

int even = 0 ← +ve

int odd = 1 ← -ve

ans

| ~~e~~ | | | | | o | e |
|---|---|---|---|---|---|---|
| 3 | -2 | 1 | -5 | 2 | -4 | 4, 6 |
| 0 | 1 | 2 | 3 | 4 | 5 | |

~~o~~ 1 7 ←

```
else // nums[i] < 0
{ ans[o] = nums[i]
  o += 2
}
```

```
if (nums[i] > 0)
{ ans[e] = nums[i]
  e += 2
}
```

# Max no of 1's wali row

| | 0 | 1 | 2 | 3 | |
|---|---|---|---|---|---|
| 0 | 1 | 0 | 0 | 0 | → 1's → 1 |
| 1 | 0 | 1 | 1 | 0 | → 1's → 2 |
| 2 | 0 | 1 | 1 | 0 | → 1's → 2 |
| 3 | 1 | 1 | 1 | 0 | → 1's → $\boxed{3}$ |
| 4 | 0 | 0 | 1 | 0 | → 1's → 1 |

oneCount = INT_MIN ?

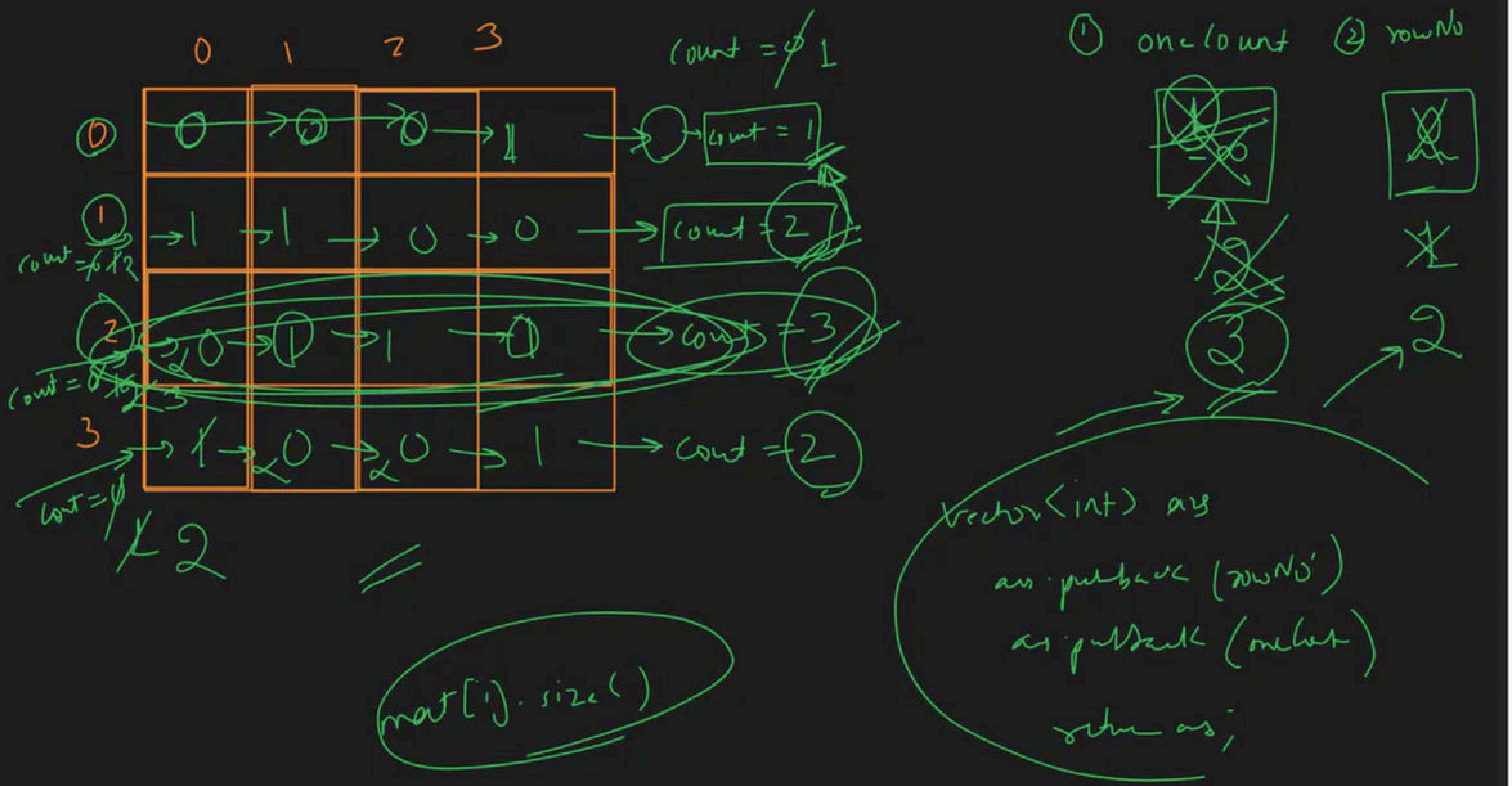
3

rowNo = -1

jab bhi naya max oneCount milega, tab rowNo store krlunga

{3, 3}

rowNo oneCount



0 1 2 3
count = 1
count = 1
count = 2
count = 3
count = 2
① oneCount ② rowNo
vector<int> ans
ans.push_back(rowNo)
ans.push_back(oneCount)
return ans;
mat[i].size()

```
for ( int i = 0; i < mat.size(); i++)
{
    for ( int j = 0; j < mat[i].size(); j++)
    {

    }
}
```

mat

row → mat.size()

row k andar kitne block h

ya

kitne columns hai

h

kaun row → ith row

mat[i]

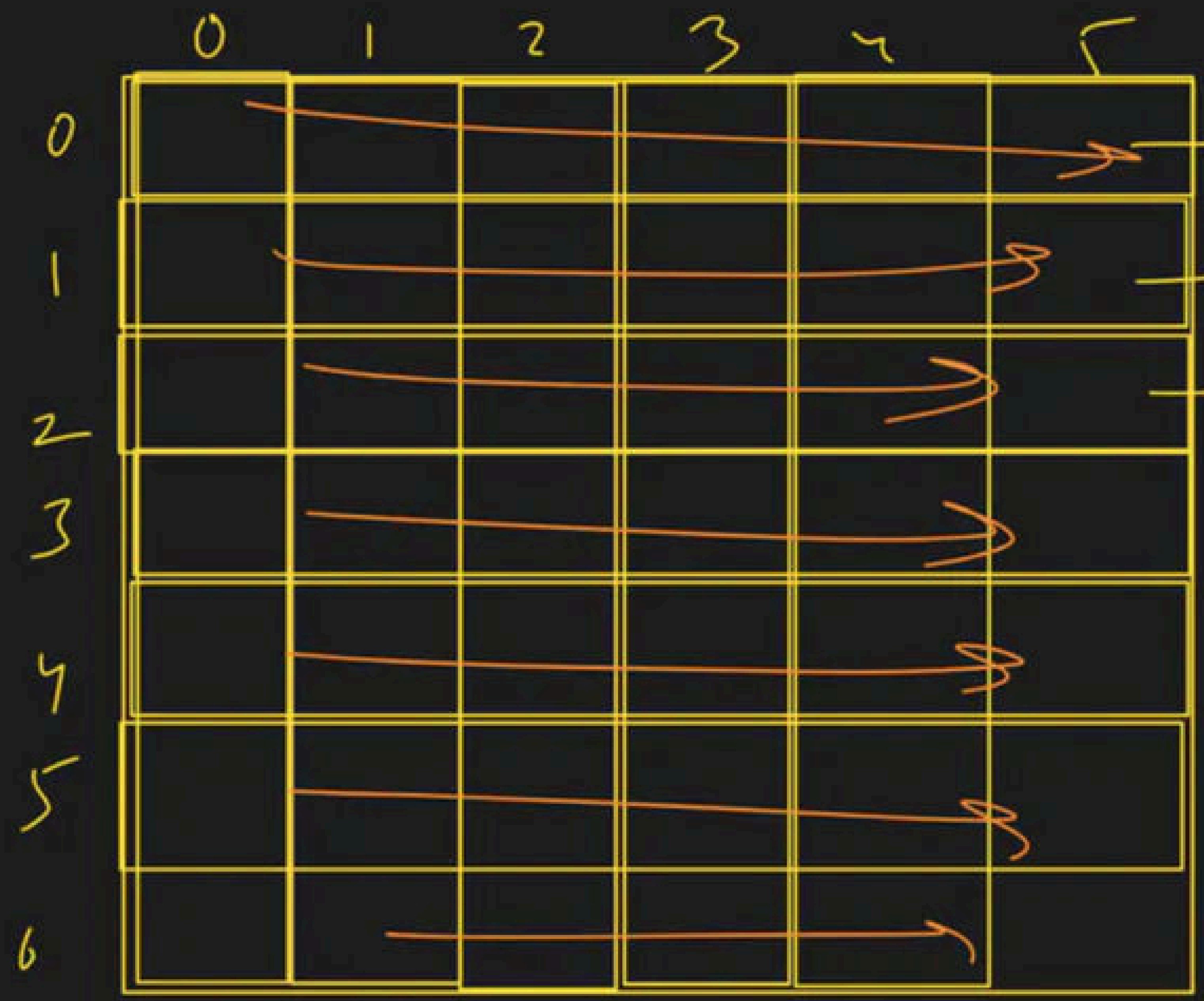
mat[i].size() ← col count

# → Rotate a 2D Array by 90 degree

arr → transpose (A) → reverse (B) → 90 degree (arr)

$$\begin{array}{c|ccc} & 0 & 1 & 2 \\ \hline 0 & 1 & 2 & 3 \\ 1 & 4 & 5 & 6 \\ 2 & 7 & 8 & 9 \end{array} \xrightarrow{\text{transpose}} \begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 9 \end{bmatrix} \xrightarrow{\text{reverse}} \begin{array}{c|ccc} & 0 & 1 & 2 \\ \hline 0 & 7 & 4 & 1 \\ 1 & 8 & 5 & 2 \\ 2 & 9 & 6 & 3 \end{array}$$

matrix




→ reverse → matrix(0)

→ —— → matrix[1]

→ row → matrix (2)

$i^{th}$ row →

matrix[i]

for (i=0 → i<n)
{
reverse → matrix(i)
}

1D vector <int> arr

matrix[i]

reverse (matrix[i].begin(), matrix[i].end())

reverse ( arr.begin() , arr.end() )

int arr[n]

reverse( arr , arr + n )

[redacted]

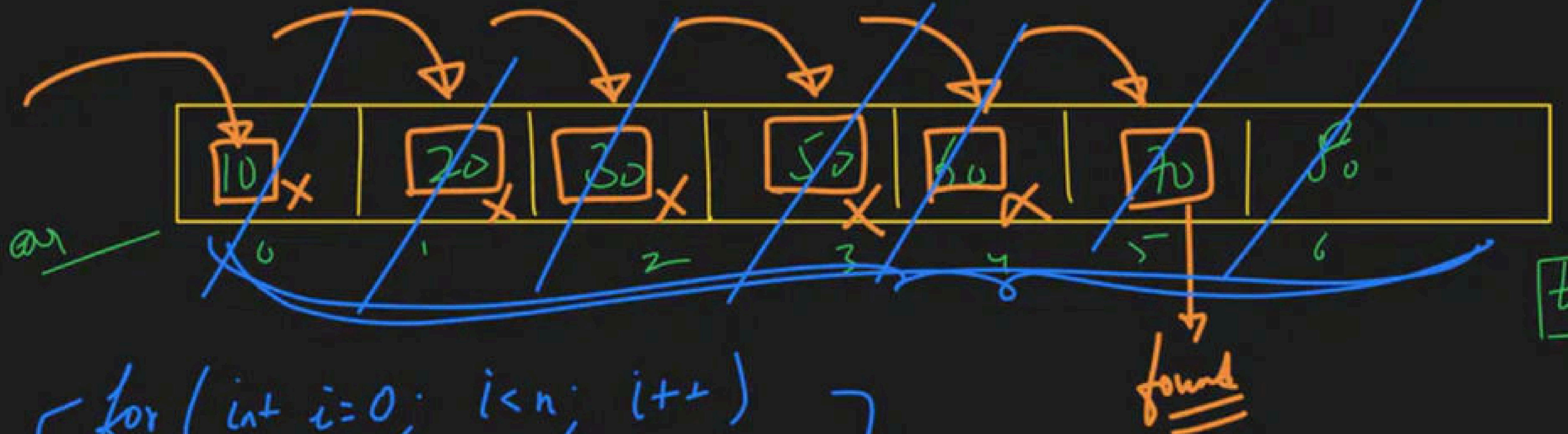
# Searching && Sorting - Level 1

Special class

Love Babbar · Sept 13, 2023

# Linear Search → 1-1 Krke sabko check krdungi

vector <int> arr;




target = 70

```
for (int i=0; i<n; i++)
{
    if (arr[i] == target)
        return true;
}
return false
```

L.S → T.C → $O(n)$

↳ size of array

⇒ Binary Search $\xrightarrow{\text{cond}^n}$ array → sorted → asc
→ desc

array → ~~~~ monotonic function

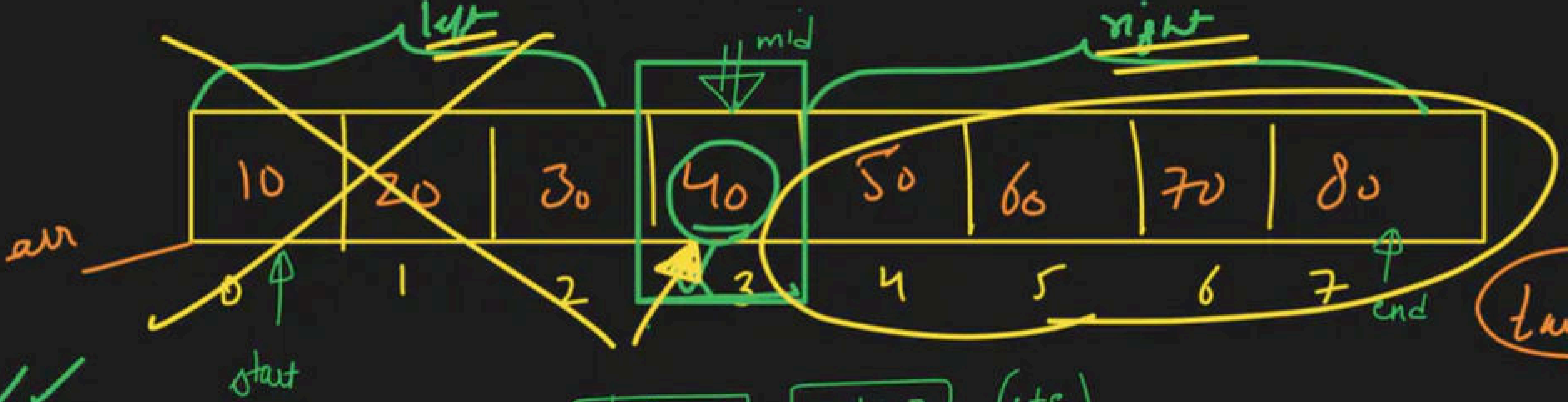
ascending

| | | | |
|---|---|---|---|
| 2 | 4 | 6 | 8 |

descending

| | | | |
|---|---|---|---|
| 10 | 9 | 7 | 2 |

Reload if no awaz




$target = 70$

(A) s, e

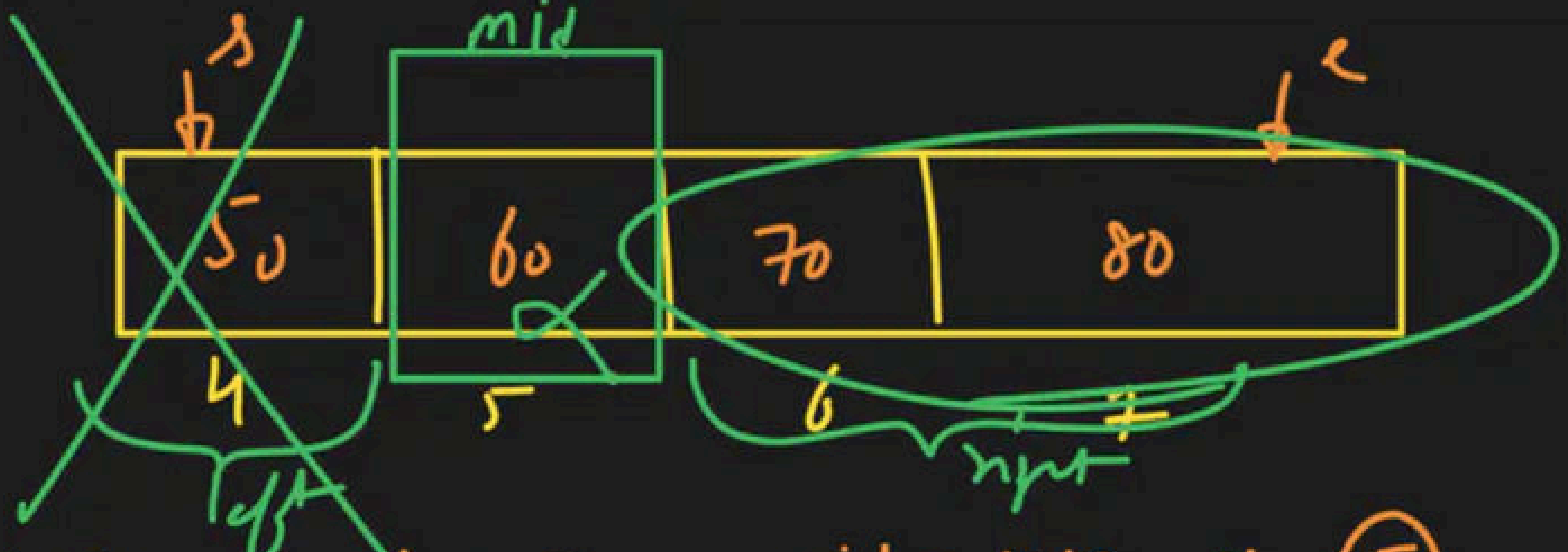
(B) $mid = \frac{(s+e)}{2}$

$\boxed{start = 0}$, $\boxed{end = 7}$ $\left(\frac{s+e}{2}\right)$

$$mid = \frac{0+7}{2} = \frac{7}{2} = 3$$

$$40 == 70 \rightarrow F$$

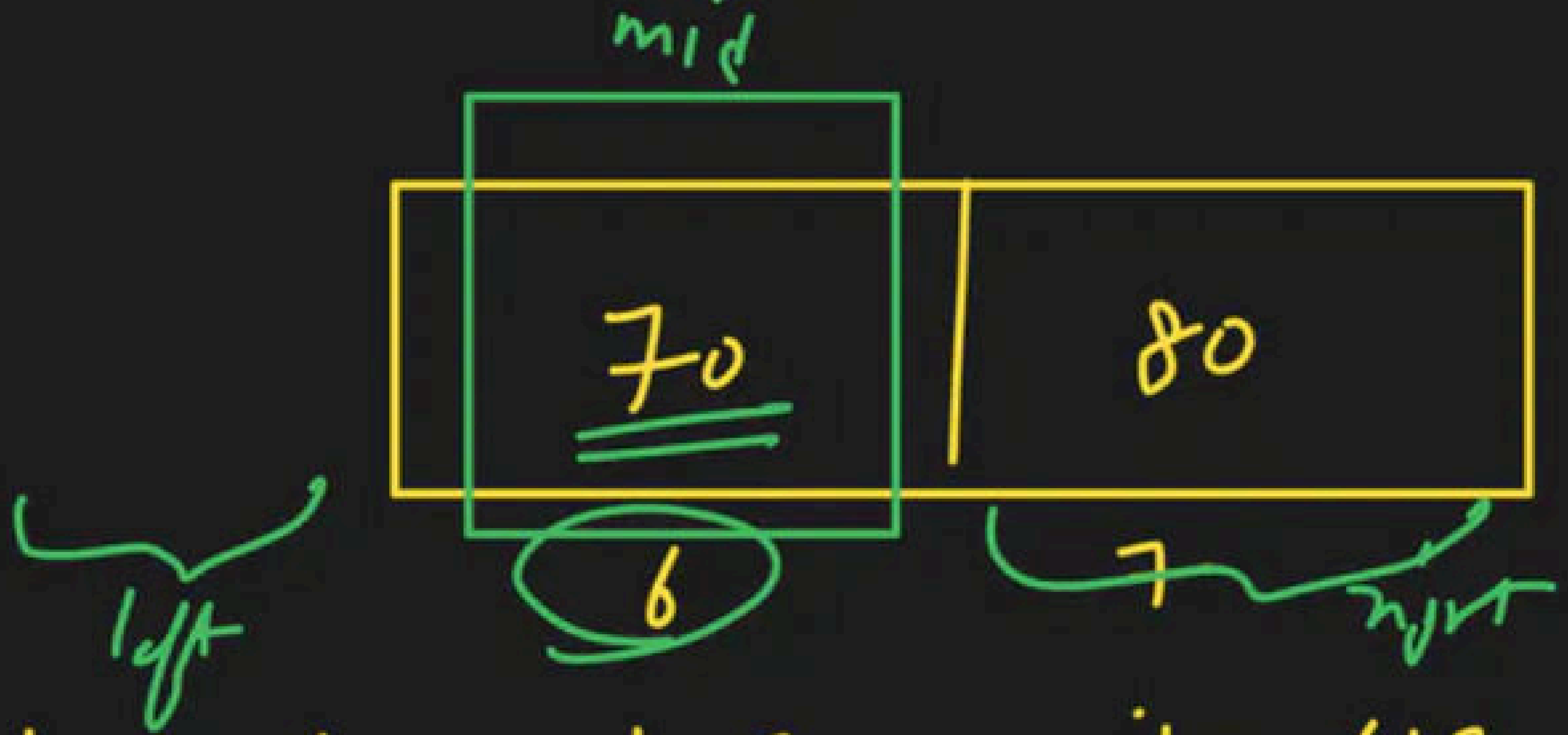
$$70 > 40 \longrightarrow \text{right}$$




(A) start = 4 , end = 7 , mid = $\frac{4+7}{2} = \frac{11}{2} = 5$

$60 == 70 \rightarrow F$

$70 > 60 \rightarrow$ Right $\rightarrow$

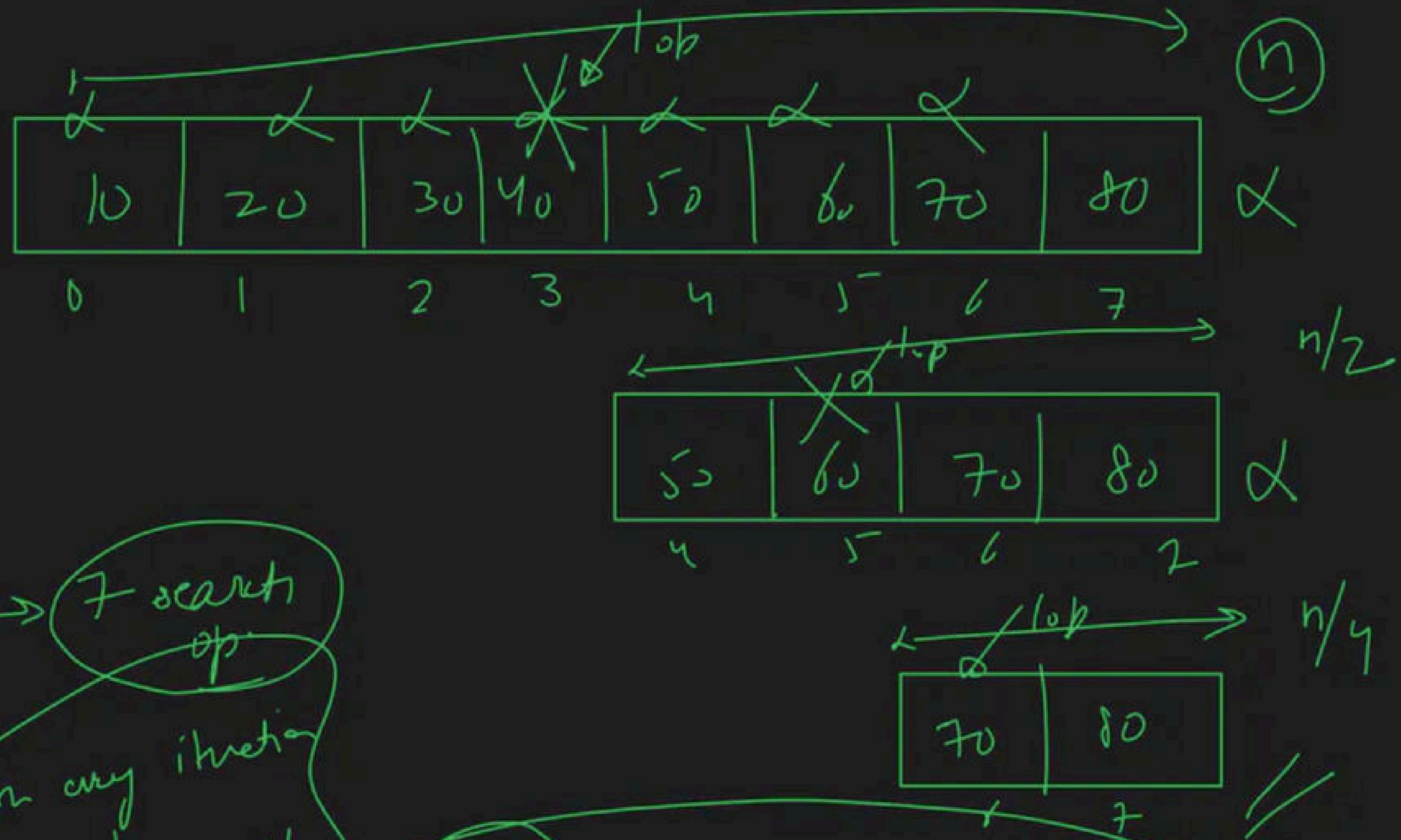



(4) start = 6 , end = 7 , $mid = \frac{6+7}{2} = \frac{13}{2} = 6$

70 == 70 → yes → True

return index

return 6




L.S → 7 search op.

OBS → with every iteration ↓ array size half

B.S

3 op → found/not found

array → 10000 element

L.S

10000 comparision

Binary search

I^st → 10000

II^nd → 5001

III → 2500

→ 1250

625

312

156

78

39

19

9

4

2

1

13

10000000

$\log_2(10000000)$

$70 > 40$

$n \text{ size} \rightarrow \dfrac{n}{2^0}$ $\quad \dfrac{n}{1}$

T.C $\rightarrow$

$n/2 \rightarrow \dfrac{n}{2^1}$

$n/4 \rightarrow \dfrac{n}{2^2}$

L.S $\rightarrow O(n)$

B.S $\rightarrow O(\log n)$

K iteration

$$\frac{n}{2^k} = 1$$

$$\boxed{n = 2^k}$$

$$\boxed{\log_2 n = K}$$

T.C $\rightarrow O(\log n)$

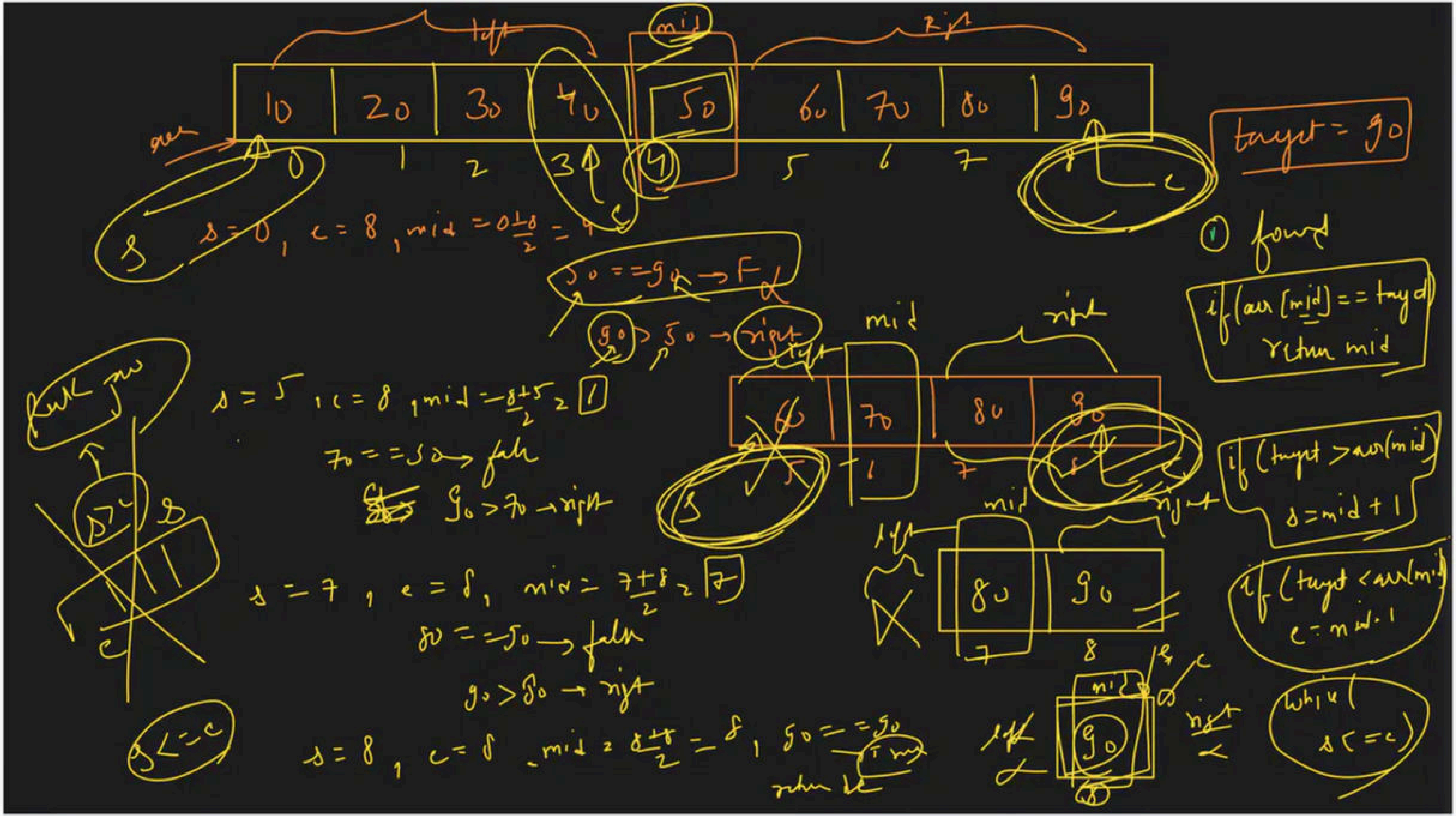
$$n = 2^k$$

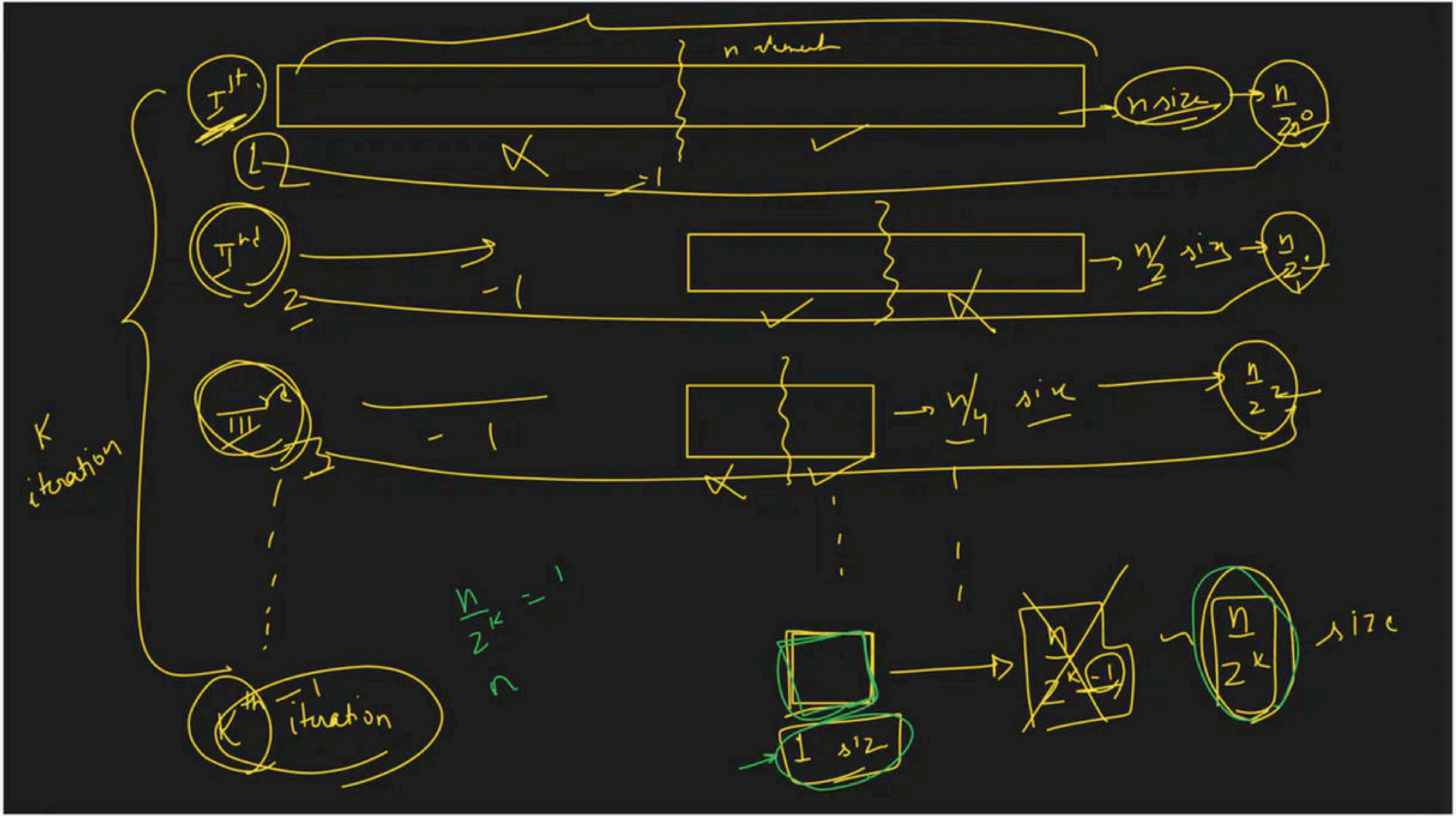
$$\log_2 n = \log_2 (2^k)$$

$$\boxed{\log n = K}$$

1 size

$\rightarrow \cancel{\dfrac{n}{2^{k-1}}}$ $\quad \boxed{\dfrac{n}{2^k}}$


left
mid
right
10 20 30 40 50 60 70 80 90
0 1 2 3 4 5 6 7 8
arr
target = 90
s = 0, e = 8, mid = (0+8)/2 = 4
50 == 90 → F
90 > 50 → right
s = 5, e = 8, mid = (8+5)/2 = 6
70 == 90 → false
90 > 70 → right
s = 7, e = 8, mid = (7+8)/2 = 7
80 == 90 → false
90 > 80 → right
s = 8, e = 8, mid = (8+8)/2 = 8, 90 == 90
true
return 8
s <= e
1 found
if (arr[mid] == target)
return mid
if (target > arr[mid])
s = mid + 1
if (target < arr[mid])
e = mid - 1
while (
s <= e
60 70 80 90
80 90
90



n element
Ist
1
-1
n size
n/2^0
IInd
2
-1
n/2 size
n/2^1
IIIrd
-1
3
n/4 size
n/2^2
K iteration
K th iteration
n/2^K = 1
n
1 size
n/2^K - 1
n/2^K size

$\frac{n}{2^k} = 1$

$$\frac{a}{b} = c$$

$n = 1 \times \underline{\underline{2^K}}$

$a = c \times b$

$n = 2^K$ → $K = ?$

$1 \times a \to \boxed{a}$

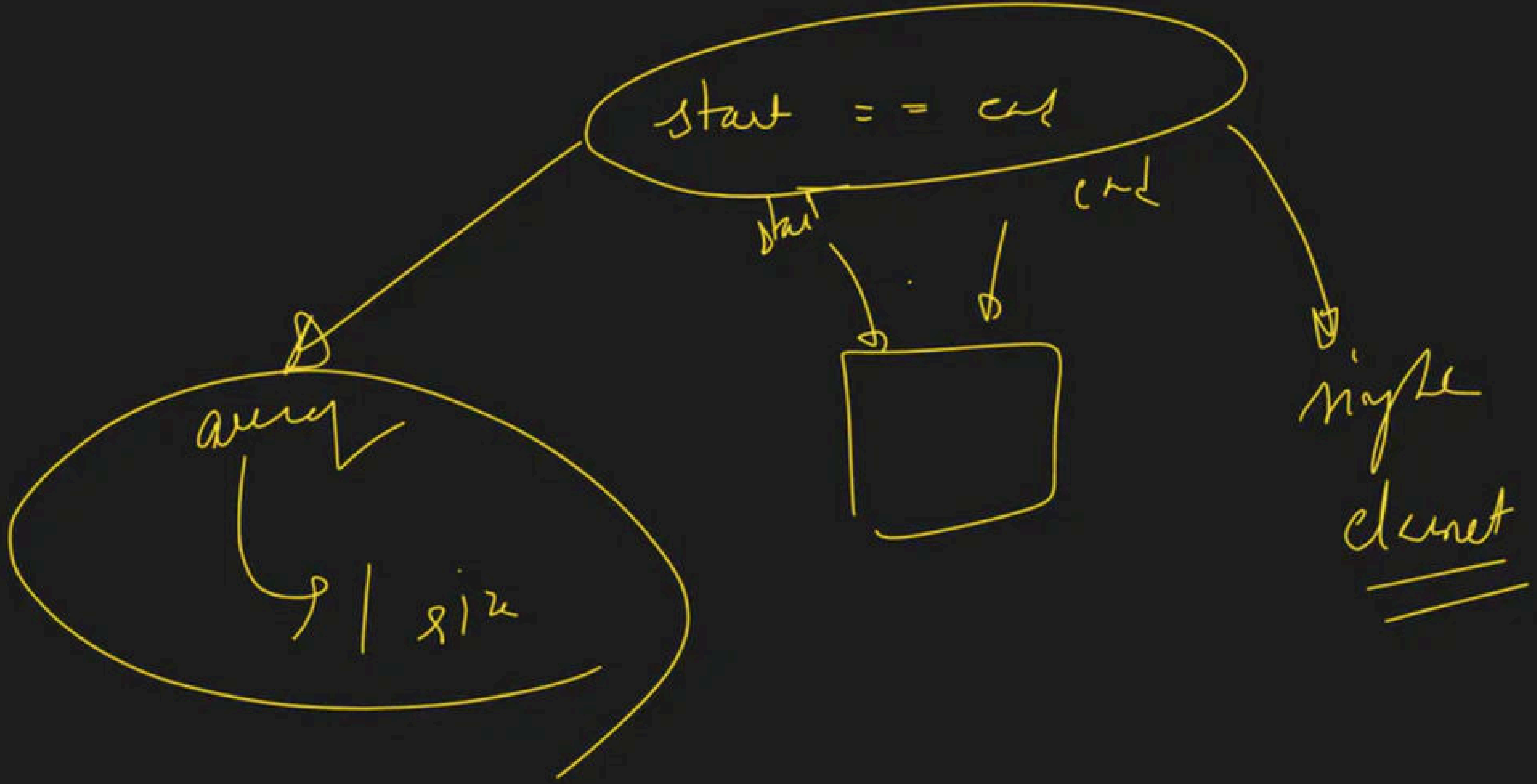
apply both side → $\log_2$

$\log_2 n = \log_2 (2^K)$

T.C → $O(K)$

$\log_2 n = K$

$O(\log_2 n)$


start == end
start
end
array
1 size
single
element

→ find first occurrence of a number in a sorted array

target = 30

ans = 2

mid

arr →

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 10 | 20 | 30 | 30 | 30 | 30 | 40 | 50 |
| 0 | 1 | $\boxed{2}$ | 3 | 4 | 5 | 6 | 7 |

first occurrence

→ s = 0 , e = 7 , mid = $\boxed{3}$   arr[mid] == target

30 == 30 → True → found → ans = 3

left → e = mid − 1

2 min → socho

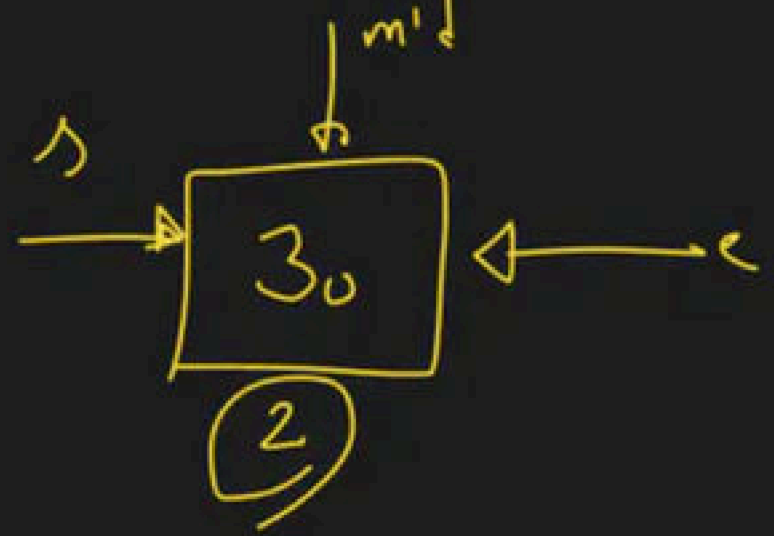
found → ans store → left me chalo jao

| | | |
|---|---|---|
| 10 | 20 | 30 |
| 0 | 1 | 2 |

→ s = 0 , e = 3 − 1 , e = 2 → $\boxed{\text{mid} = 1}$

20 == 30 → X

30 > 20 → ✓ → right → s = mid + 1 = 1 + 1 , s = 2




$$\text{int mid} = s + \frac{(e-s)}{2}$$

$s = 2$, $e = 2$, $mid = \frac{2+2}{2} = 2$

arr[mid] == target

30 == 30 → found → ans = 2

→ left me jao

$e = mid - 1$

$e = 2 - 1$

$e = 1$

→ $s = 2$, $e = 1$ → $s > e$ → break karo

→ find last occurrence → sorted array

target = 30

mid

arr

| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|
| 10 | 20 | 30 | 30 | 30 | 30 | 30 | 40 | 50 |

s = 0 → mid = 4
e = 8

arr[mid] == target
30 == 30 → found

found → Ans store
right me chale jao → s = mid+1

find → 4-5 → T.C

Dry Run

find total occurrence

target = 30

| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|
| 10 | 20 | 30 | 30 | 30 | 30 | 40 | 50 |

Total Occ → 4

first Occ = 2

last Occ = 5

$\boxed{\text{Total Occ} = \text{last Occ} - \text{first Occ} + 1}$

$\boxed{\text{B.S}}$ → Advanced Question

2 min → Paani Break

$$\text{int mid} = \cancel{\frac{(s+e)}{2}}$$

INT_MAX

+

INT_MAX

$$\cancel{\frac{s+e}{2}}$$

$$s + \frac{(e-s)}{2}$$

$$= s + \frac{(e-s)}{2}$$

$$s + \frac{(e-s)}{2} \rightarrow s + \frac{e}{2} - \frac{s}{2}$$

$$\rightarrow \frac{s}{2} + \frac{e}{2} \rightarrow \frac{s+e}{2}$$

→ find missing element in a sorted array

$1 \to n$

arr →

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 6 | 7 | 8 | 9 |
| 0 +1 | 1 +1 | 2 +1 | 3 +1 | 4 +2 | 5 +2 | 6 +2 | 7 +2 |

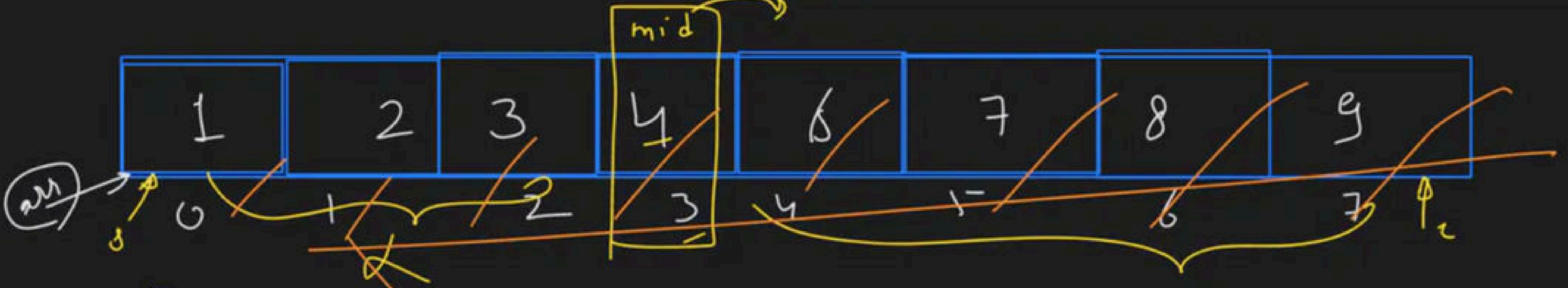
$\underbrace{\qquad\qquad\qquad}_{+1 \text{ pattern}}$

pattern break

ans = 5

missing

B.S → 1 or 2 → 2 min




$s = 0$
$e = 7$ → $mid = \frac{0+7}{2} = 3$

$diff = arr(mid) - mid$
$= 4 - 3$
$diff = 1$

$\boxed{diff == 1}$ → right → $s = mid + 1$

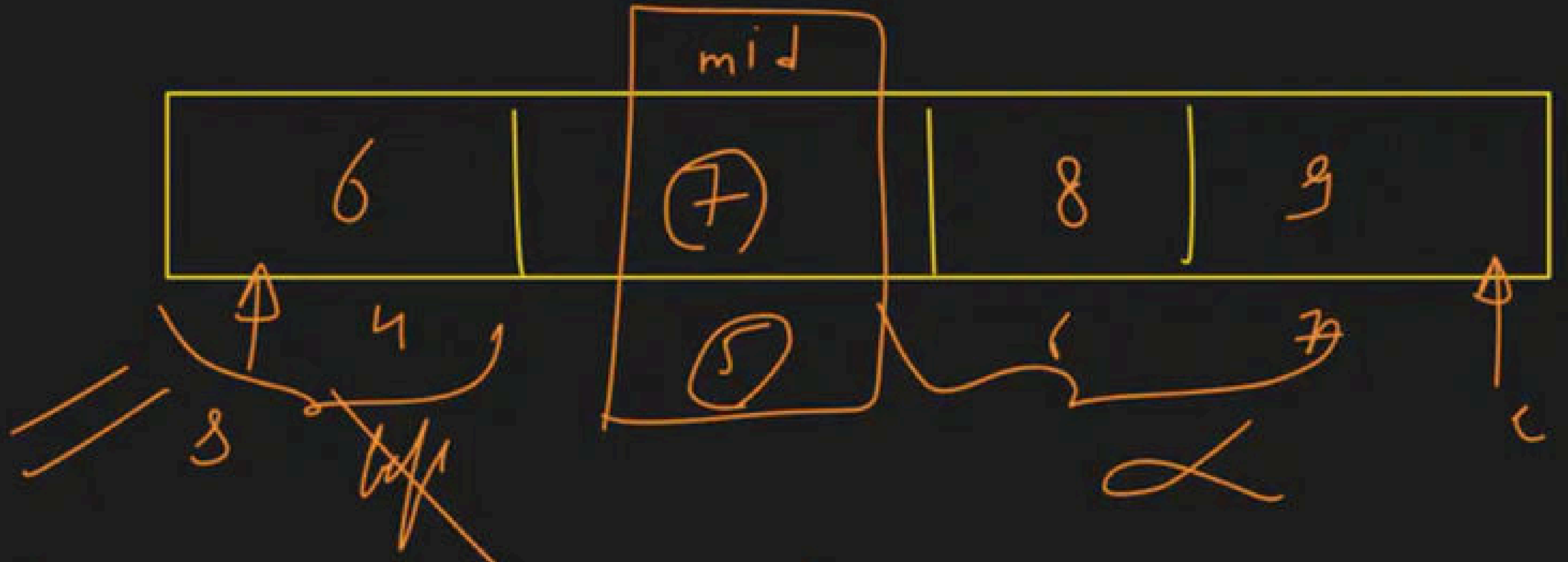
i/p → (n - 1) integer

& integer

1 → n

1 — 9

1 → 9




$s = 4$

$e = 7$ $\rightarrow$ $mid = \frac{4+7}{2} = 5$

$diff = arr[mid] - mid$

$= 7 - 5$

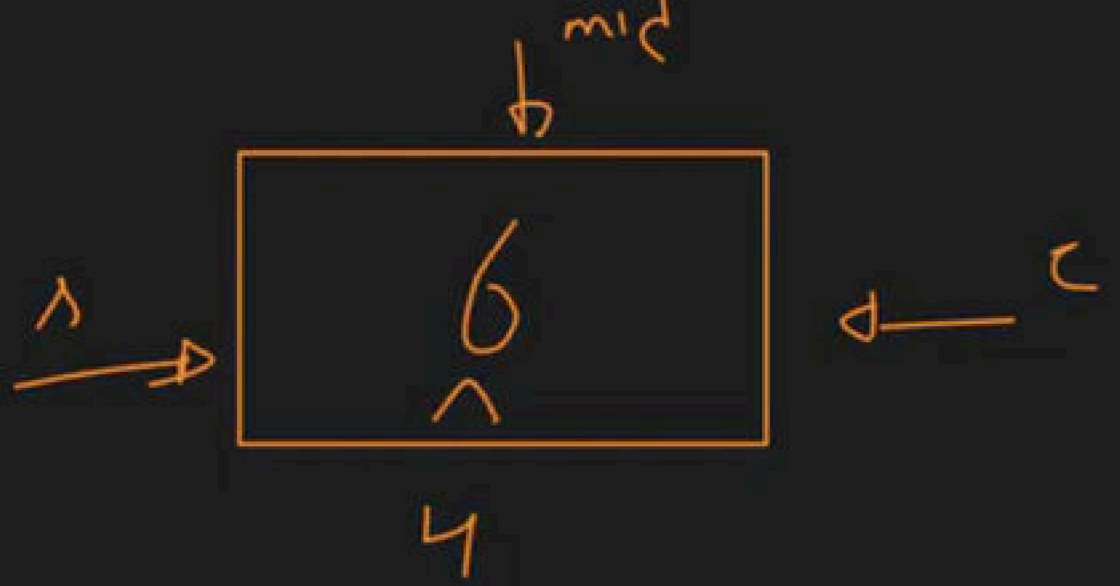
$diff = 2$

$diff == 1 \rightarrow$ false $\rightarrow$ ans store

$ans = 7$

$\rightarrow$ left me jao

$e = mid - 1$




$s = 4$, $e = 4$ → $mid = \frac{4+4}{2} = 4$

$diff = arr[mid] - mid$

$= 6 - 4$

$diff = 2$

$diff == 1$ → No → $\boxed{ans = 4}$

left → $e = mid - 1$

$s = 4$, $e = 3$ → $s > e$ → break jao

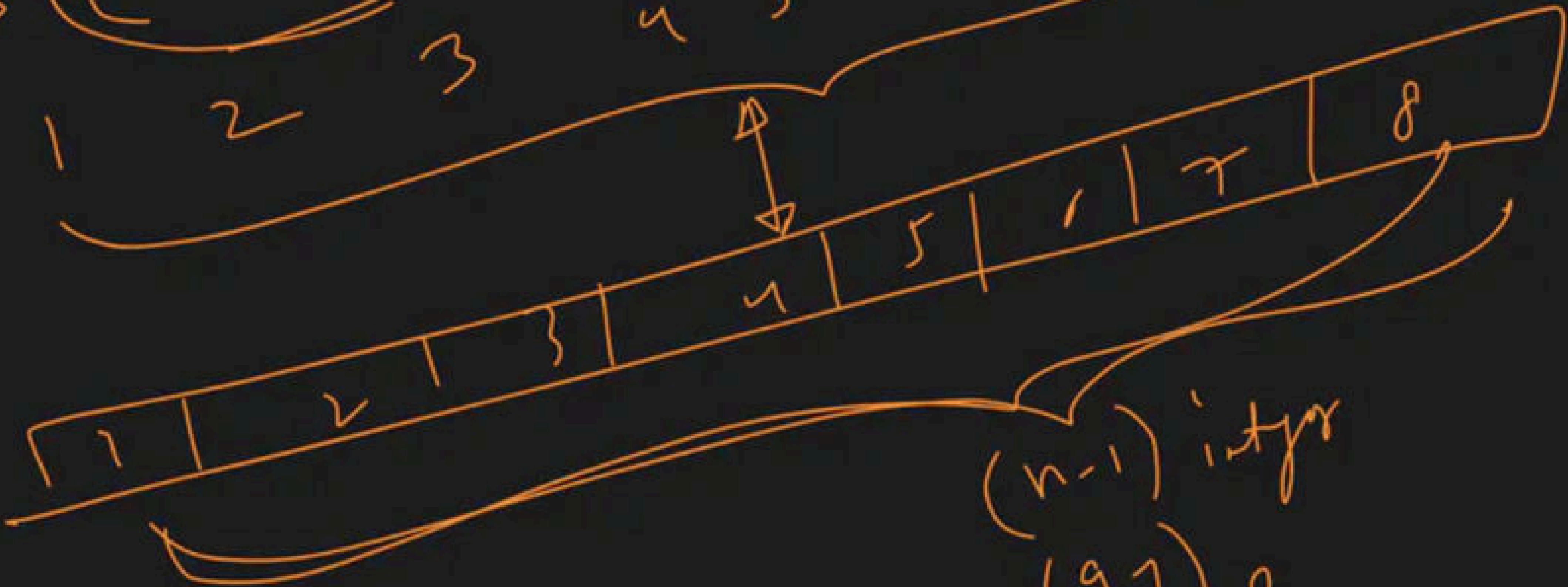
findAns → ans + 1

(n+1)

$1 \longrightarrow n$

$1 \longrightarrow 9$

1 2 3 4 5 6 7 8 9




(n-1) integer

(9-1) 8

# Peak element in a Mountain Array

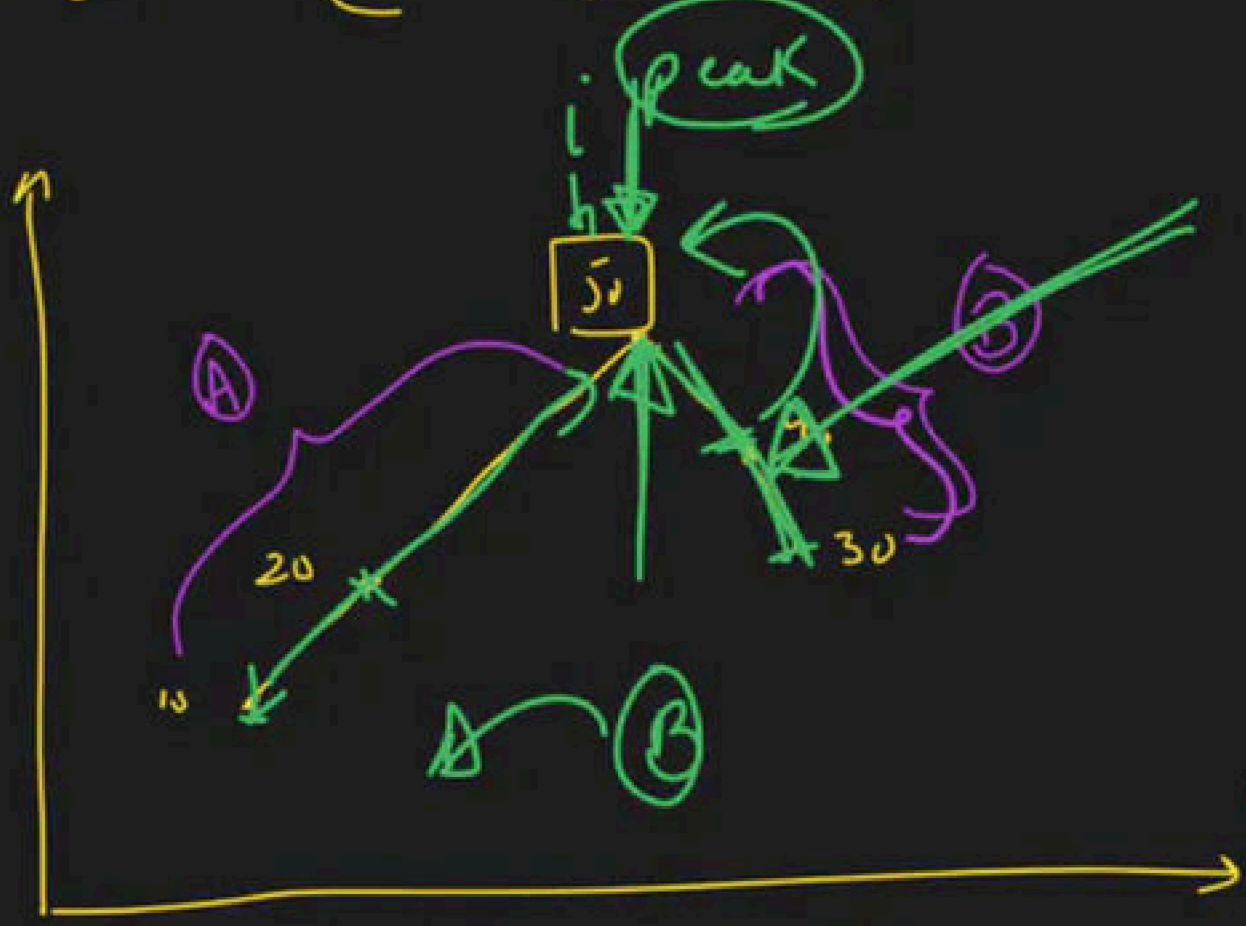
i/p → array( ) = { 10, 20, 50 40 30 }

A

arr[i] < arr[i+1]

B

arr[i] > arr[i+1]




ans = 50 → o/p

Kaise → ?

peak point

arr[i-1] < arr[i] > arr[i+1]

Peak element in Mountain

Code

By Lakshay

Debug

$arr[i] < arr[i+1]$ → A

~~arr~~

NOT can

B + Peak

Observation:-

A

$arr[i] < arr[i+1]$

B

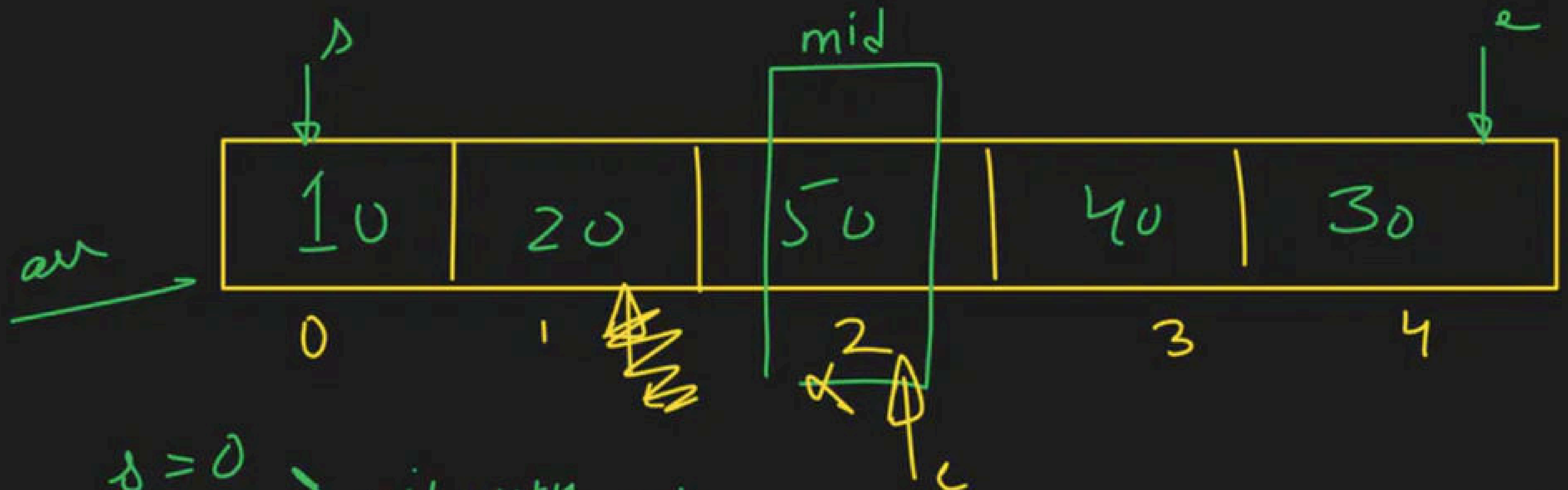
$arr[i] > arr[i+1]$

Peak

we can club

$arr[i] > arr[i+1]$

$arr[i] > arr[i-1]$



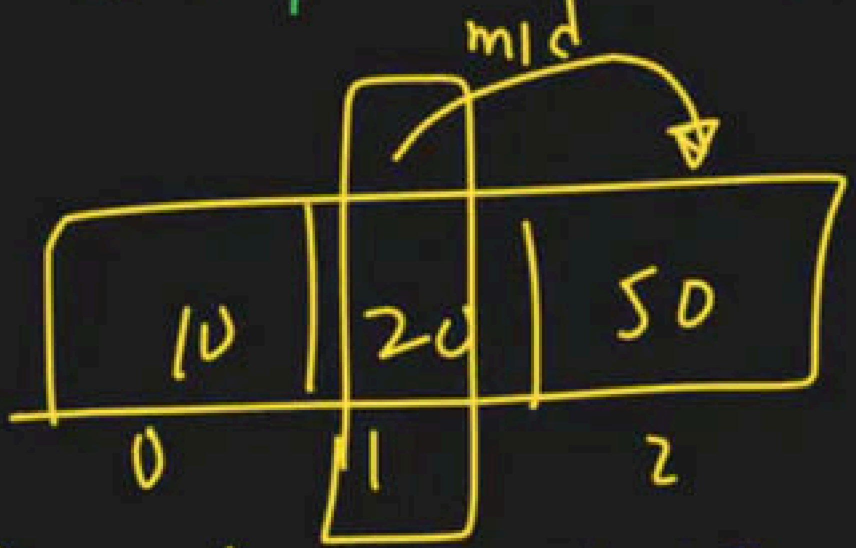

$s = 0$, $e = 4$ } $mid = \frac{0+4}{2} = 2$

arr[mid] < arr[mid+1]

50 < 40 → $\boxed{\text{false}}$ → B or peak

left

last nahi hai




$s = 0$, $e = 2$ } - mid = 1

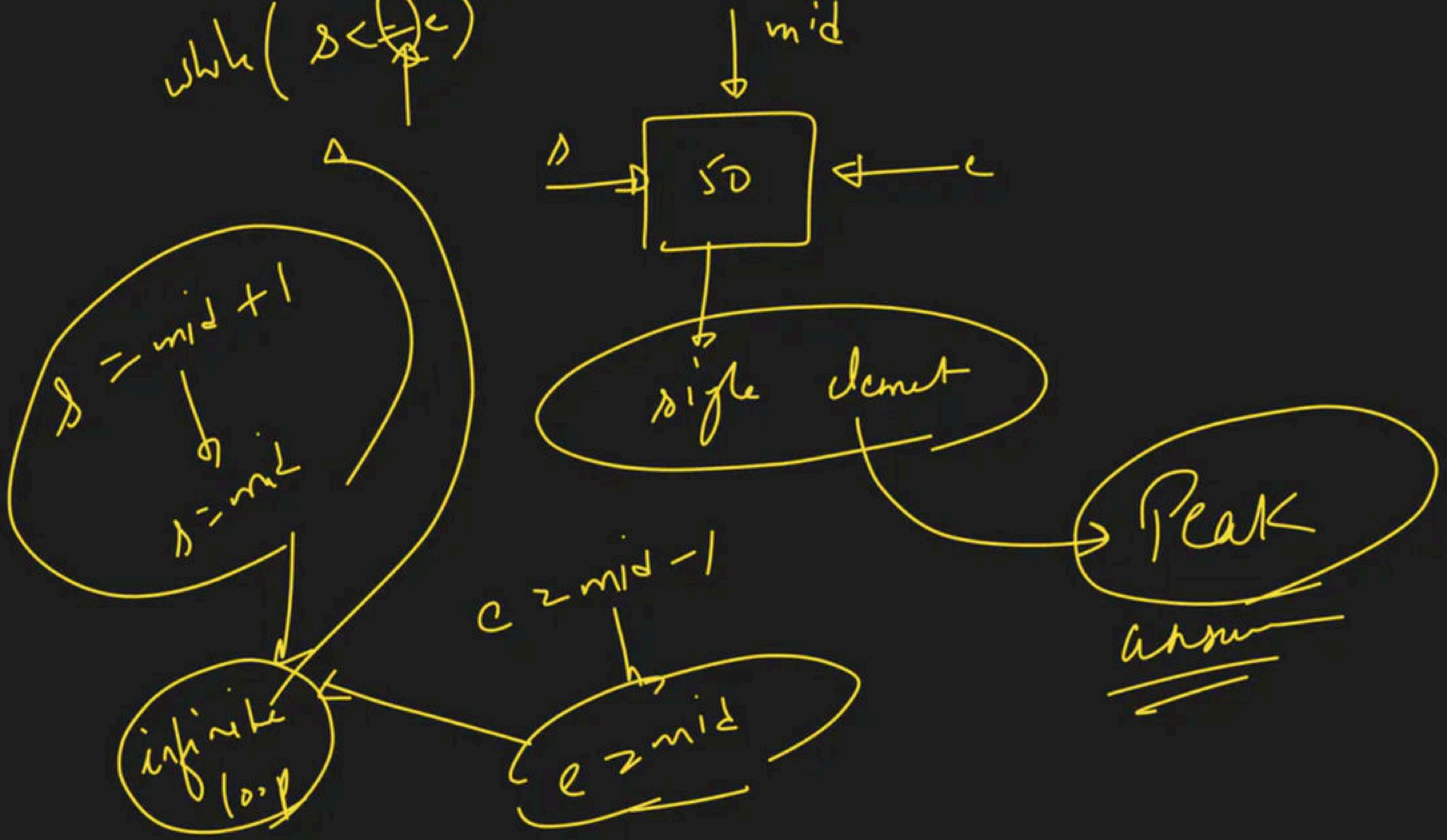
20 < 50 → True → Right.

e = mid - 1

✗

✓ e = mid → $\boxed{e = \cancel{mid - 1}}$ ← left

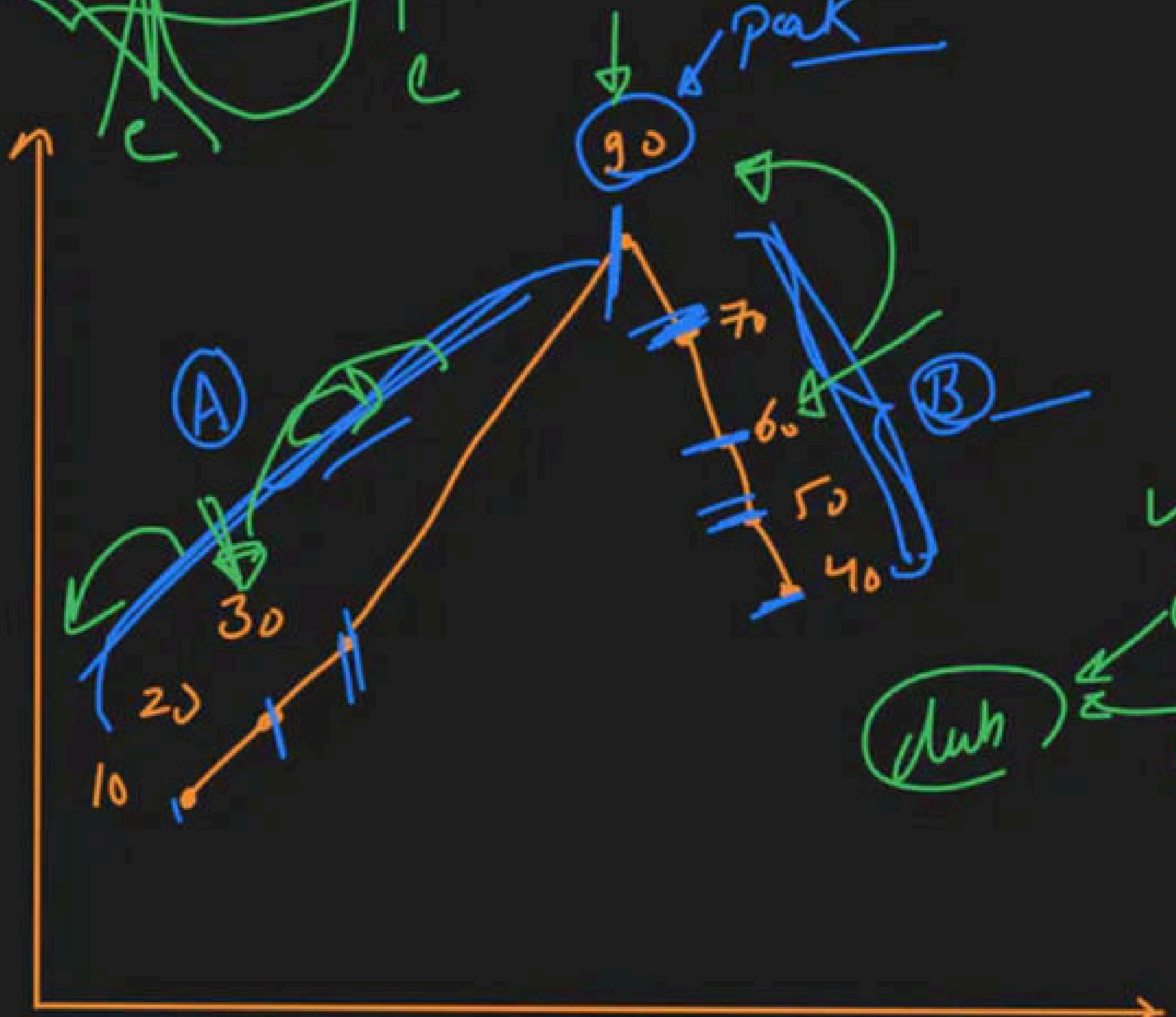
e = mid



while( s<=e )
mid
s
SD
e
single element
Peak
answer
s = mid+1
s = mid
e = mid-1
e = mid
infinite loop

| arr | | | | | | | |
|---|---|---|---|---|---|---|---|
| 10 | 20 | 30 | 90 | 70 | 60 | 50 | 40 |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |

s e m e




A element → right

B → left

Peak → wahin pr rehta

A: arr[i] ≤ arr[i+1] → True

B: arr[i] > arr[i+1]

ans ← Peak

arr[i] > arr[i+1]

arr[i] > arr[i-1]

s < e

s <= e

```
if (arr[mid] < arr[mid+1])
{   //right
        s = mid + 1
}
else
{
    //left
        e = mid;
}
```

→ true → A

B or Peak

s ~~e~~

arr

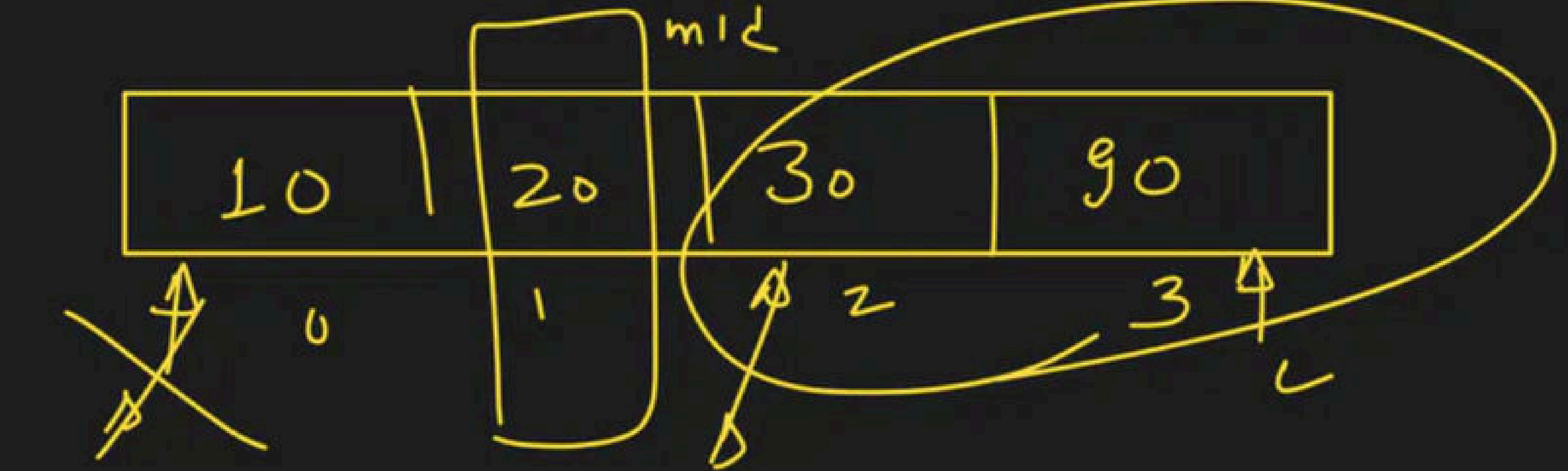
| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 10 | 20 | 30 | 90 | 70 | 60 | 50 | 40 |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |

$s = 0$

$e = 7$ — mid → $\frac{0+7}{2} = 3$

arr[mid] < arr[mid+1]

$90 < 70$ → false → $e = \text{mid}$

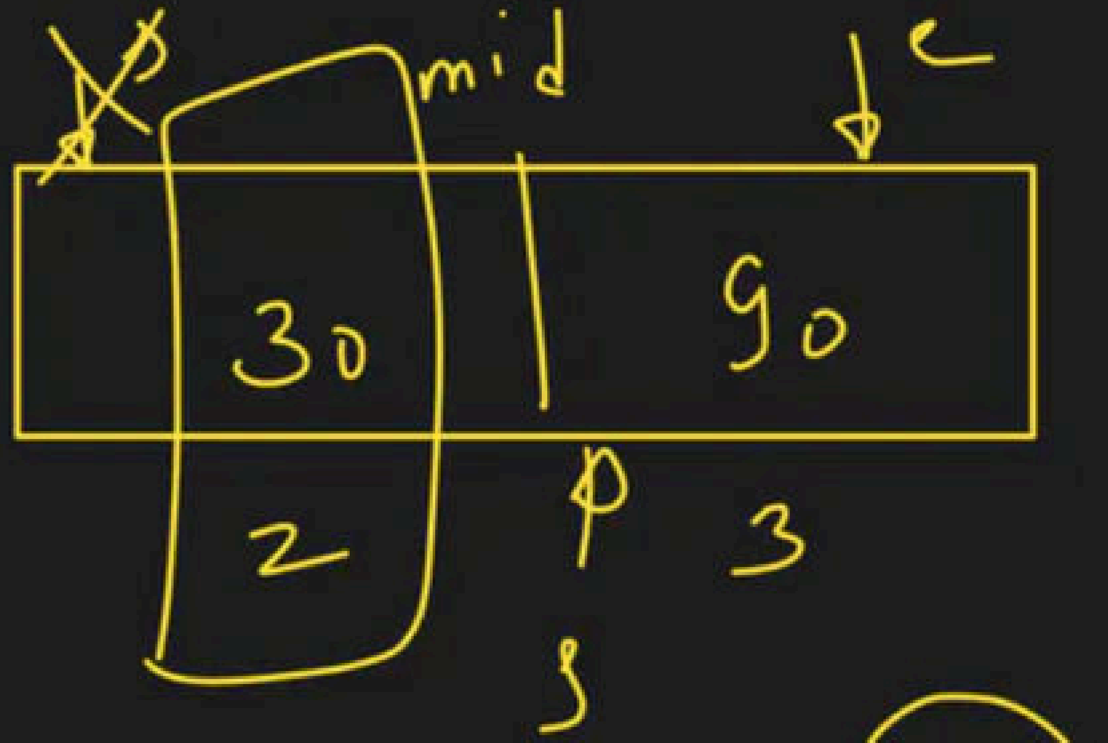



$s = 0$, $e = 3$ — $mid = \frac{0+3}{2} = 1$

$arr[mid] < arr[mid+1]$

$20 < 30 \rightarrow$ true

$\rightarrow$ Right




$s = 2$

$e = 3$

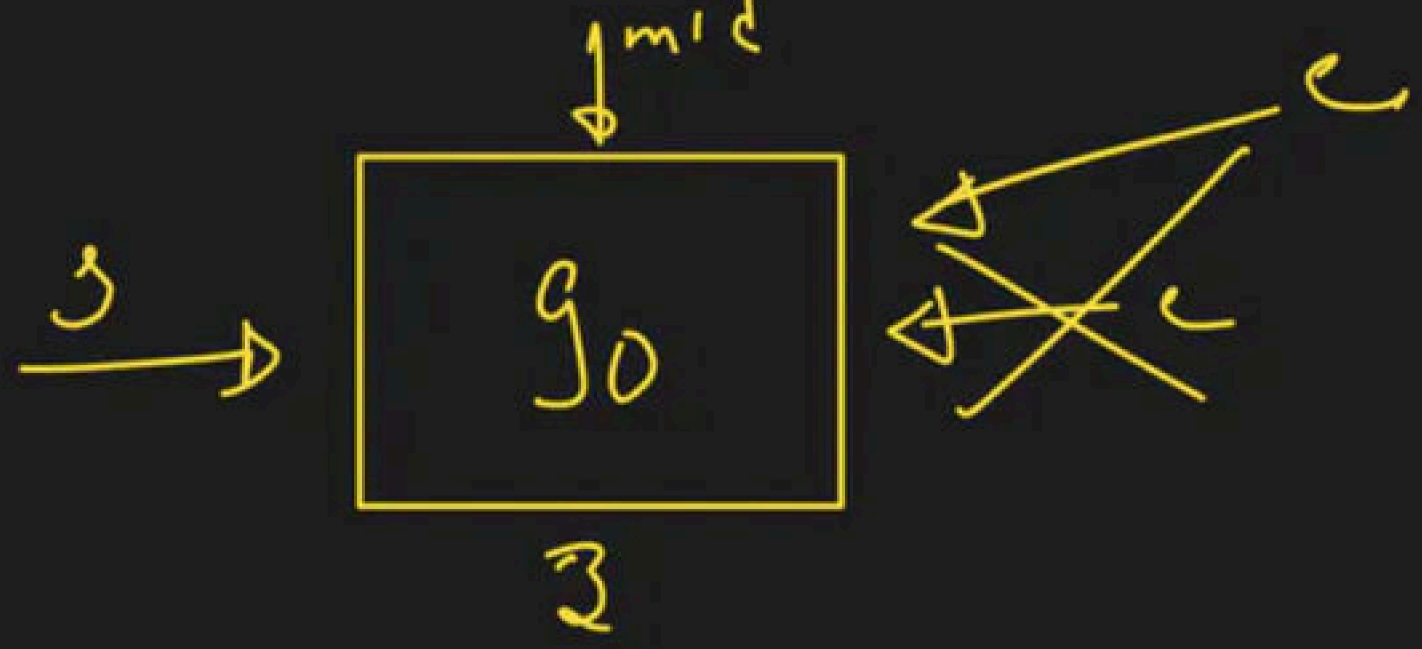
$mid = \frac{2+3}{2} = 2$

$arr[mid] < arr[mid+1]$

$30 < 50 \rightarrow$ true

right

$s = mid+1$




$s = 3$

$e = 3$

$mid = \frac{3+3}{2} = 3$

$arr(mid) < arr[mid+1]$

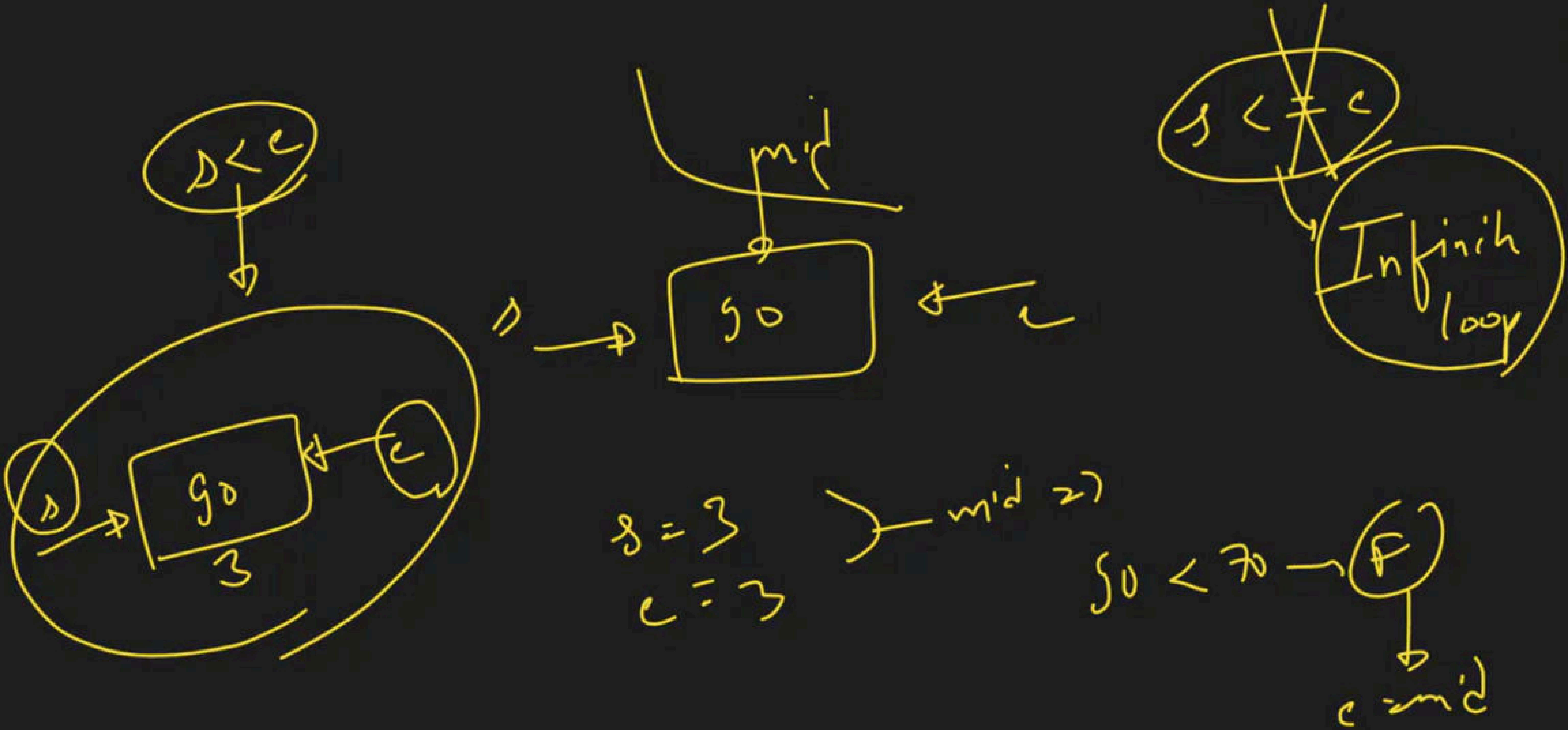
$90 < 70 \rightarrow$ false

$e = mid$


s = 3
e = 3

→ mid = 3 → go < 70 → F

e = mid



s<e
go
s
e
3
mid
go
s
e
s<=e
Infinite loop
s = 3
e = 3
mid 2)
go < 70
F
e = mid

find pivot element

GFG